

ARM PrimeCell

PS2 Keyboard/Mouse Interface (PL050)

Design Manual

ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Design Manual

© Copyright ARM Limited 1999. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell PS2 Keyboard/Mouse Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	KMI DESIGN OVERVIEW	3
4.1	Functional Description	3
4.2	Signal Description	4
4.3	Programmer's Model	5
4.3.1	Functional Mode Registers	5
4.3.2	Test Mode Registers	6
4.3.3	Register Memory Map	7
4.4	Block Partitioning	8
4.4.1	APB Interface	8
4.4.2	Register Block	10
4.4.3	Bit Counter	11
4.4.4	Transmit logic	12
4.4.5	Receive Logic	13
4.4.6	KMI Controller	14
4.4.7	Timer	15
4.4.8	Reference Clock Divider	16
4.4.9	KMICLKIN Synchroniser	16
4.4.10	Synchronisers	17
5	KMI IMPLEMENTATION DETAILS	18
5.1	Design Hierarchy	18
5.2	Kmi Top Level Module (Kmi)	18
5.3	Kmi APB Interface (KmiApbif)	18
5.3.1	Write Interface	19
5.3.2	Read Interface	19
5.4	KMI Reg Block (KmiRegBlk)	20
5.4.1	KMITXINTR signal generation	20
5.4.2	KMIRXINTR signal generation	20
5.4.3	Synchronisation of the KMICLKDIV registers to KMIREFCLK domain	21

5.5	Generation of the 8MHz pulse (KmiREFCLKDiv)	21
5.6	KMICLKIN Synchroniser (KmiClkInSync)	22
5.7	Transmitter/Receiver logic (KmiTxRx)	22
5.7.1	Bit Counter (KmiBitCounter)	23
5.7.2	Transmit Section (KmiTx)	23
5.7.3	Receive Section (KmiRx)	24
5.8	KMI Controller state machine (KmiCntrl)	24
5.9	Timer section (KmiTimer)	28
5.10	Synchronisers to KMIREFCLK domain (KmiSynctoKMIREFCLK)	29
5.11	Synchronisers to PCLK domain (KmiSynctoPCLK)	29
5.12	Test logic (KmiTest)	30
6	KMI TEST DETAILS	31
6.1	KMI Testbenches	31
6.1.1	VHDL Bus Compliance Testbenches	31
6.1.1.1	Unit Under Test	35
6.1.1.2	APB Bridge Model	35
6.1.1.3	Trickbox	35
6.1.1.4	Protocol Checker	35
6.1.1.5	APB Multiplexer	35
6.1.1.6	Vector Sets	36
6.1.1.7	Generics	36
6.1.1.8	Running Simulations	37
6.1.2	Verilog Bus Compliance Testbenches	39
6.1.2.1	Unit Under Test	43
6.1.2.2	APB Bridge Model	43
6.1.2.3	Trickbox	43
6.1.2.4	Protocol Checker	43
6.1.2.5	APB Multiplexer	43
6.1.2.6	Vector Sets	44
6.1.2.7	Parameters	44
6.1.2.8	Running Simulations	45
6.1.3	Unification of BusTalk and TicTalk	46
6.1.4	Vector Generation	47
6.1.5	AMBA TICTalk testbenches	48
6.2	KMI Trickbox	50
6.2.1	Trickbox Signal Description	50
6.2.2	Trickbox functional description	52
6.2.3	Trickbox Register Description	53
6.3	Free-running Functional Verification Tests	57
6.3.1	Register Tests	57
6.3.2	Transmission Test	57
6.3.3	Reception Tests	57
6.3.4	Transmission Series Test	57
6.3.5	Reception Series Test	58

6.3.6	Transmission Reception Alternating Test	58
6.3.7	Transmission Reception Random Test	58
6.3.8	Receive State Test	58
6.3.9	Transmit State Test	59
6.3.10	Error Tests	59
6.3.11	Timeout Tests During Reception	59
6.3.12	Timeout Tests During Transmit	59
6.3.13	Scan Mode Test	60
6.4	Clock-enabled Production Tests	61
6.4.1	Reset Value Tests	61
6.4.2	Pattern Read / Write Tests	61
6.4.3	Read-Only Register Tests	61
6.4.4	Reset Test Using Test Reset	61
6.4.5	Timer- Counter Test	61
6.4.6	Transmit Test	62
6.4.7	Receive Test	62
6.4.8	Transmit and Receive Test	62

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	7 January, 1999	First release.
B	4 June, 1999	Second release, Verilog testbenches added.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM DDI 0143	ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual
2	PL050 INTM 0000	ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Design Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
KMI	Keyboard/Mouse Interface
RTL	Register Transfer Level
STA	Static Timing Analysis

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell PS2 Keyboard/Mouse Interface (KMI). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the KMI design, describing the functional partitioning of the KMI and the signal interconnections. Section 5 presents detailed descriptions on the KMI design with simulation waveforms. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the KMI.

3 INTRODUCTION

The KMI is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual [Ref.1]. The ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 KMI DESIGN OVERVIEW

4.1 Functional Description

The ARM PrimeCell KMI can be used to implement an IBM-compatible PS2 keyboard or mouse interface.

The KMI block performs serial-to-parallel conversion on data received from a keyboard or mouse peripheral device through the KMIDATAIN input and parallel-to-serial conversion on data received from the APB interface. The interface uses an open-drain clock line and an open-drain data line for transferring data between the host system and the keyboard/mouse peripheral. The ARM PrimeCell KMI can be configured for a 'PS2/AT' or a 'Legacy' mode (for back compatibility to a previous block) interface. The 'legacy' mode (or non-line control mode) should only be used for keyboards which are not fully PS2-compliant and do not send an acknowledge (line control) bit after receiving each data byte.

A block diagram of the device is shown in **Figure 4.1-1: KMI block diagram**.

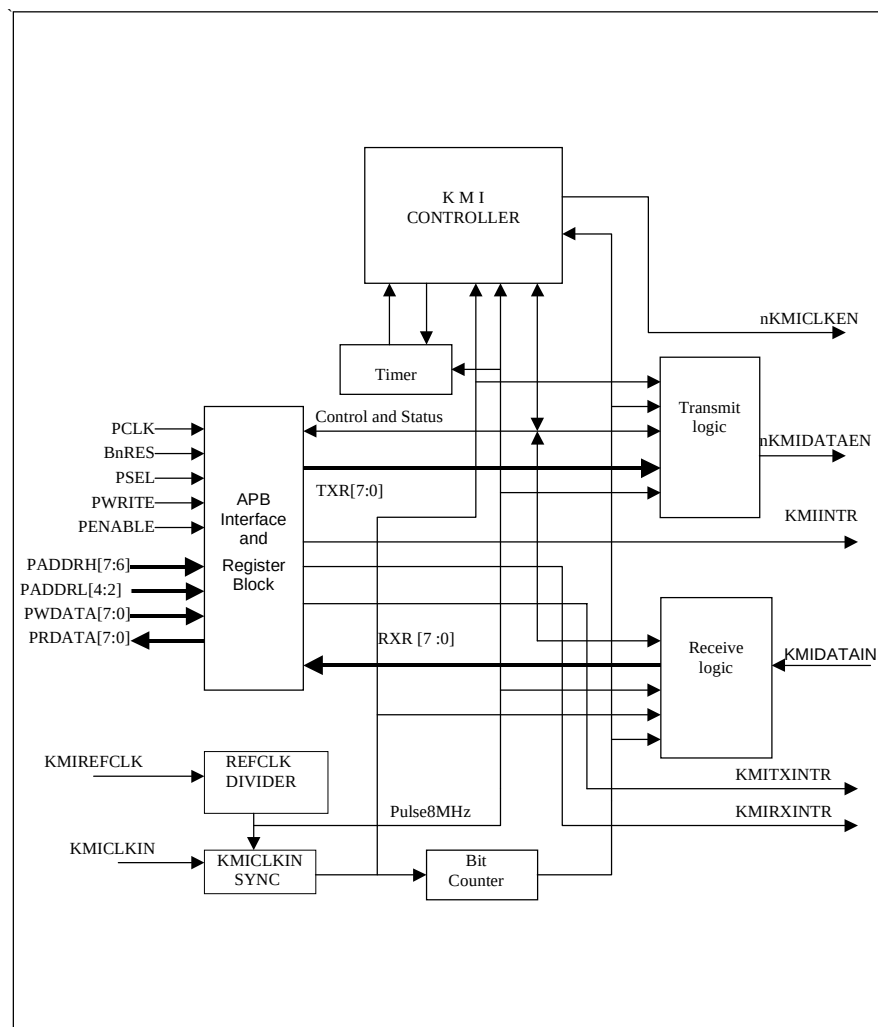


Figure 4.1-1 KMI Block Diagram

4.2 Signal Description

The following tables describe the ARM PrimeCell KMI I/O signals. **Table 4.2-1** lists the APB interface signals, **Table 4.2-2** lists the block-specific on-chip non-APB signals and **Table 4.2-3** lists the signals to I/O pad cells.

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Bus Clock input, used to time all bus transfers.
BnRES	IN	Reset Controller	Bus reset input signal (active low).
PADDRH[7:6]	IN	APB Bridge	Subset of the APB address bus.
PADDRL[4:2]	IN	APB Bridge	Subset of the APB address bus.
PWDATA[7:0]	IN	APB Bridge	Data is written to the block on this bus.
PRDATA[7:0]	OUT	APB Bridge	Data is read from the block on this bus.
PENABLE	IN	APB Bridge	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable bus transfer cycles.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write to the block and when LOW, a read from the block.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the block has been selected by the APB Bridge.

Table 4.2-1: AMBA APB signals.

Signal Name	Type	Source / Destination	Description
KMIREFCLK	IN	Clock Generator	KMI Reference clock input.
nKMIRST	IN	Reset Controller	KMI reset signal to KMIREFCLK clock domain, active LOW. The reset controller must use BnRES to assert nKMIRST asynchronously but negate it synchronously with KMIREFCLK.
KMITXINTR	OUT	Interrupt Controller	KMI Transmit interrupt (active HIGH)
KMIRXINTR	OUT	Interrupt Controller	KMI Receive interrupt (active HIGH)
KMIINTR	OUT	Interrupt Controller	KMI interrupt (active HIGH). A single combined interrupt generated as an OR function of the two individually maskable interrupts above.
SCANMODE	IN	Test Controller	KMI Scan test mode input. This signal must be asserted HIGH during scan testing to ensure that internal data storage elements can be asynchronously reset. SCANMODE must be negated LOW during normal use or when applying manufacturing test vectors via the Test Interface controller (TIC).

Table 4.2-2: On-chip signals.

Signal Name	Type	Source / Destination	Description
KMICLKIN	IN	PAD	KMI clock input.
KMIDATAIN	IN	PAD	KMI serial data input
nKMIDATAEN	OUT	PAD	KMI data pad output enable When Low – zero driven When High – High impedance
nKMICKEN	OUT	PAD	KMI clock pad output enable When Low – zero driven When High – High impedance

Table 4.2-3: Signals to I/O pad cells.

4.3 Programmer's Model

4.3.1 Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
KmiBase + 0x00	R/W	6	0x00	KMICR	Control Register
KmiBase + 0x04	R	7	0x43	KMISTAT	Reads from this register return status information.
KmiBase + 0x08	R/W	8	0x00	KMIDATA	Writes to this register will load the Transmit register. Reads from this register will return the data received.
KmiBase + 0x0C	R/W	4	0x00	KMICLKDIV	Writes to this register will load the reference clock divisor value to provide the 8MHz internal clock. Reads from this register will return the programmed value.
KmiBase + 0x10	R	2	0x02	KMIIR	Interrupt Identification Register
KmiBase + 0x14-3C					Reserved.
KmiBase + 0x40-9C					Reserved (for test purposes)
KmiBase + 0xA0-FF					Reserved.

Table 4.3-1 Functional Mode Registers

4.3.2 Test Mode Registers

Address	Type	Width	Reset Value	Name	Description
KmiBase + (0x40 to 0x7C)	R/W	0	-	KMITCER	Accesses to this virtual register will result in the creation of an internal test clock enable
KmiBase + 0x80	R/W	5	0x00	KMITCR	This register controls the clocking mode and the input pin multiplexing during test mode.
KmiBase + 0x84	R/W	4	0x00	KMITMR	This register controls the mode of the timer during test.
KmiBase + 0x88	R/W	2	0x00	KMITISR	This register provides direct stimulus control of the non-AMBA primary inputs to the device.
KmiBase + 0x8C	R	3	0x03	KMITOCR	This register provides observation of the KMI non-AMBA primary outputs.
KmiBase + 0x90	R	6	0x00	KMITIMERSTG1	This register provides observability for the Stage 1 counter of Timer.
KmiBase + 0x94	R	5	0x00	KMITIMERSTG2	This register provides observability for the Stage 2 counter of Timer and the 16ms, 64μs timeout.
KmiBase + 0x98	R	8	0x00	KMITIMERSTG3	This register provides observability for the Stage 3 counter of Timer.
KmiBase + 0x9C	R	4	0x06	KMISTATE	This register provides observability for the states of the KMI Controller FSM (Finite state machine).

Table 4.3-2 Test Mode Registers

4.3.3 Register Memory Map

Address	Read Location	Write Location
KMI Base + 0x00	KMICR	KMICR
KMI Base + 0x04	KMISTAT	-----
KMI Base + 0x08	KMIDATA	KMIDATA
KMI Base + 0x0C	KMICLKDIV	KMICLKDIV
KMI Base + 0x10	KMIIR	-----
KMI Base + (0x10 to 0x3C)	Reserved	Reserved
KMI Base + (0x40 to 0x7C)	KMITCER (Test Mode Only)	KMITCER (Test Mode Only)
KMI Base + 0x80	KMITCR (Test Mode Only)	KMITCR (Test Mode Only)
KMI Base + 0x84	KMITMR (Test Mode Only)	KMITMR (Test Mode Only)
KMI Base + 0x88	KMITISR (Test Mode Only)	KMITISR (Test Mode Only)
KMI Base + 0x8C	KMITOCR (Test Mode Only)	-----
KMI Base + 0x90	KMITIMERSTG1 (Test Mode Only)	-----
KMI Base + 0x94	KMITIMERSTG2 (Test Mode Only)	-----
KMI Base + 0x98	KMITIMERSTG3 (Test Mode Only)	-----
KMI Base + 0x9C	KMISTATE (Test Mode Only)	-----

Table 4.3-3 Register Memory Map

4.4 Block Partitioning

The sub-modules in the design are listed below.

- APB Interface (Input module and Output module)
- Register block
- Transmit logic
- Receive logic
- Timer
- Bit Counter
- Reference Clock Divider
- Kmi Controller
- Kmi Clock Synchroniser
- Synchronisers

The design of the above-mentioned sub-modules is detailed below.

4.4.1 APB Interface

The APB interface comprises an input and output module. This interface deals with reads and writes to registers from the APB. A simplified block diagram of the APB interface block is shown in **Figure 4.4-1**.

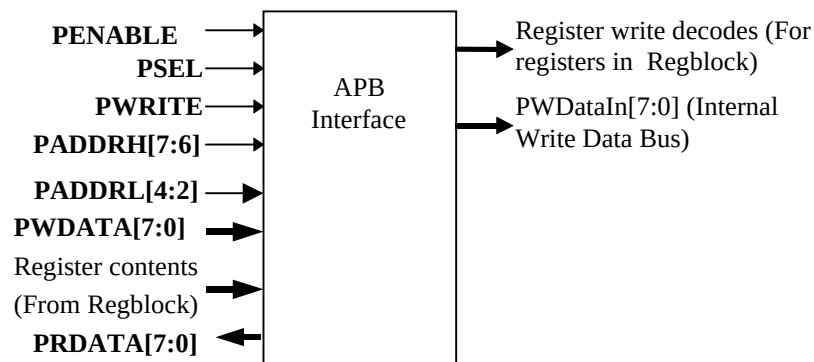


Figure 4.4-1 APB Interface Block diagram

Input Module

This module provides the APB write interface. It generates decodes for writes to all registers in the device. Writes to the KMIDATA register actually result in writes to the transmit register. Each register addressable from the APB is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode and PENABLE, PSEL and PWRITE signals.

Non-AMBA inputs such as KMIDATAIN, KMICLKIN, are multiplexed with their corresponding test register bits. The block diagram in **Figure 4.4-2** illustrates the structure of the input module. In this block diagram, the write decodes are generated in the APB interface module, while the registers are actually located in the register block, RegBlock.

Since KMIREFCLK is the main clock for the KMI core logic, control register bits are synchronised to KMIREFCLK before use in the KMI core.

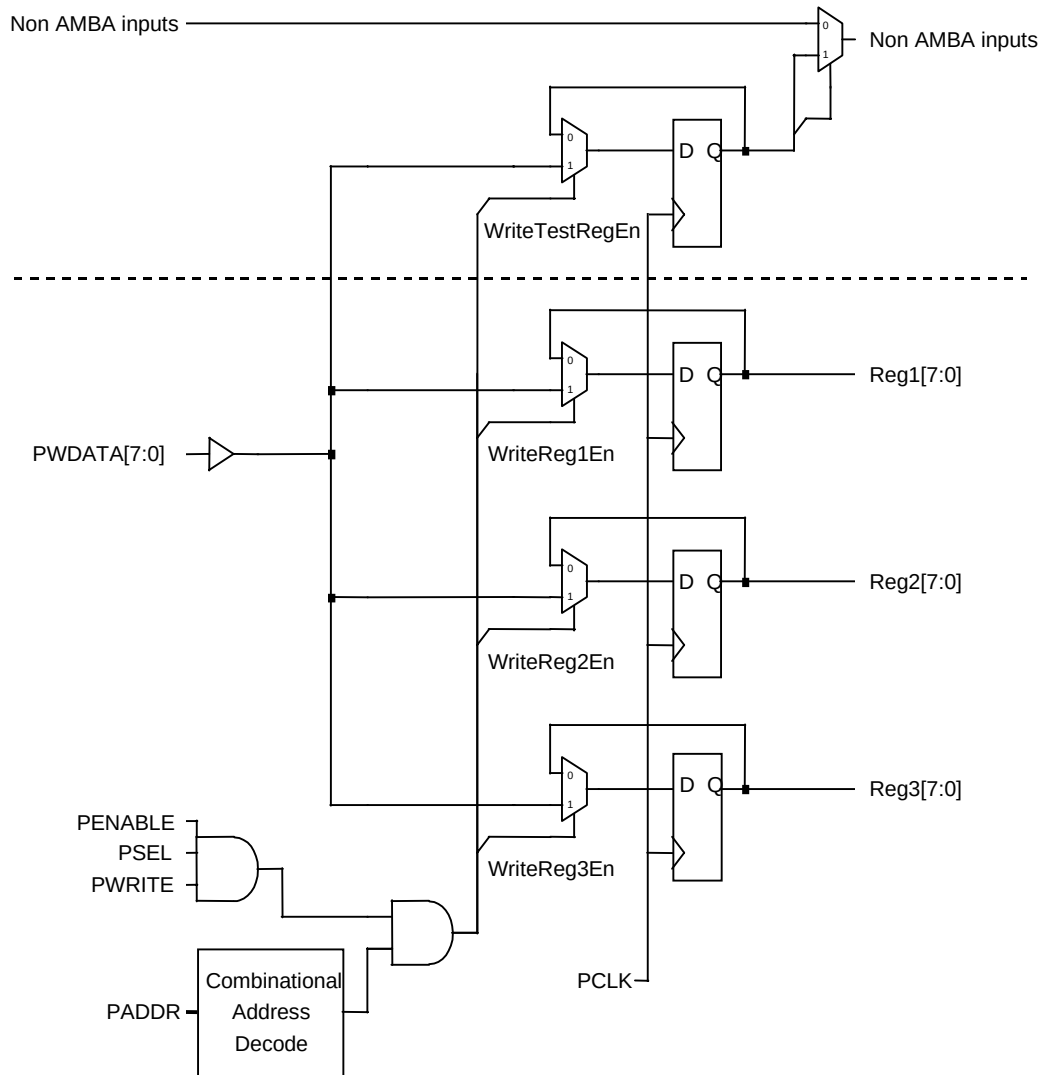


Figure 4.4-2 Input module

Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read data bus. The output of the multiplexer is fed to a bank of flip-flops clocked by the system PCLK. A combinatorial address decode is used to select which of the output sources are driven onto the bus. The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of external bus connection method used for the data bus on the APB. A block diagram of the output module is shown in **Figure 4.4-3**. The registers shown in this block diagram are actually located in the register block, while the output multiplexer and the final data register on the output of the multiplexer are located in the APB interface block.

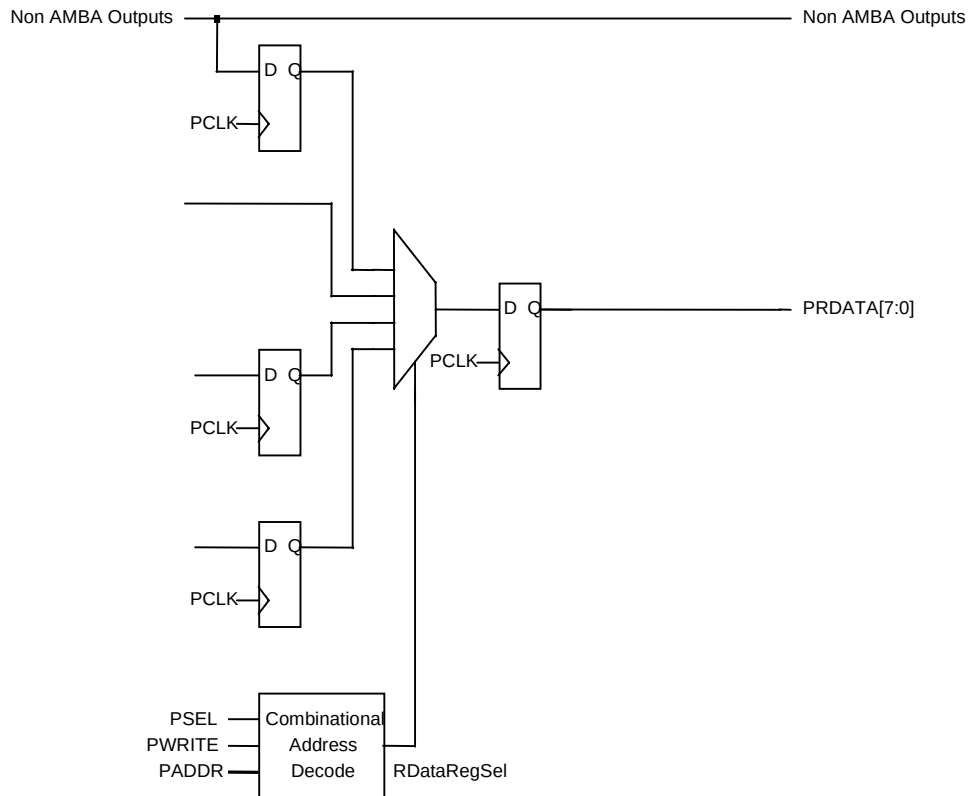


Figure 4.4-3 Output Module

4.4.2 Register Block

Besides the registers themselves, this block also contains the logic required to generate the KMITXINTR and KMIRXINTR. It also asserts the TxEmpty flag to indicate that a byte can be written to KMIDATA register for transmission and the RxFull flag to indicate that a byte has been received in the KMIDATA register.

A two-buffer technique is used to synchronise the contents of the KMICLKDIV register to the KMIREFCLK domain.

In this 2-buffer technique, writes from the APB to the KMICLKDIV register, go to the respective first stage buffers. When there is a write to KMICLKDIV, a control signal toggles. This signal is double-synchronised to the KMIREFCLK domain. An edge on the synchronised signal is detected by generating a delayed version and XOR-ing the synchronised signal with its delayed version. Thus, a one-KMIREFCLK-wide load pulse is generated with every write to the KMICLKDIV register. When this load signal is asserted, the contents of the first stage KMICLKDIV buffer is copied into the second stage buffer in the KMIREFCLK domain.

Figure 4.4-4 illustrates the 2-buffer mechanism.

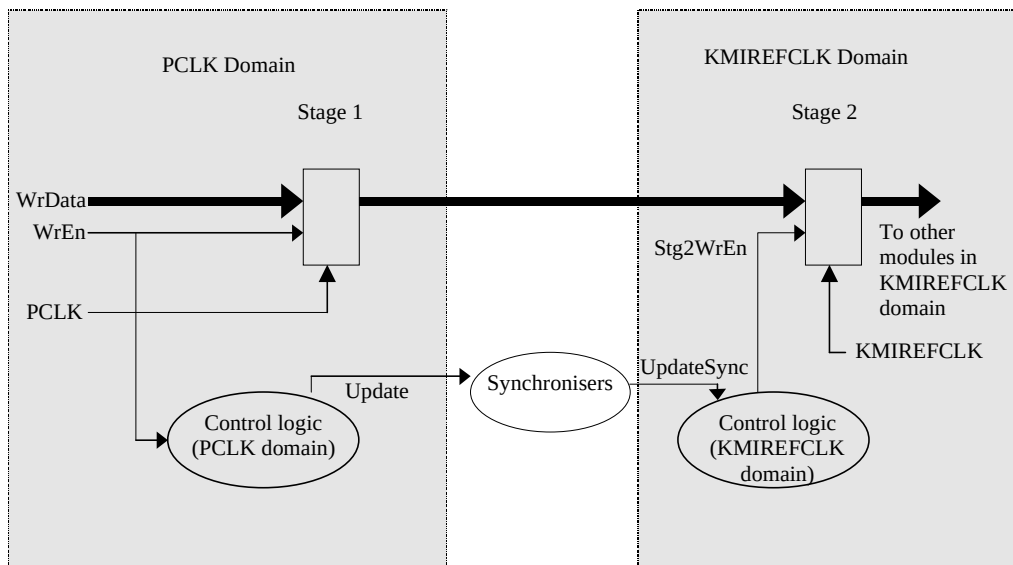


Figure 4.4-4 Two-buffer synchronisation mechanism for KMICLKDIV

When synchronisation is in progress, it is expected that there are no further writes immediately to the KMICLKDIV register. This is a reasonable assumption since this register is a prescaler register and is not expected to be written frequently.

4.4.3 Bit Counter

The bit counter contains one 4-bit up-counter, clocked by KMIREFCLK. The bit counter keeps track of the number of bits transmitted or received. The bit counter is reset by the KMI controller at the start of transmission or reception. The bit counter is incremented on every Pulse8MHz signal on seeing the KmiClkInLow signal asserted, provided the nKMICKEN is not being driven by the controller. **Figure 4.4-5** shows the block diagram of the Bit Counter.

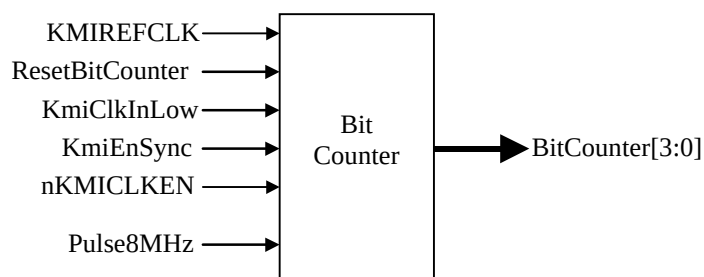


Figure 4.4-5 Bit Counter

4.4.4 Transmit logic

The transmit logic converts the parallel transmit data into a serial bit stream with a bit rate dependent on the incoming KMI clock signal. This block performs odd parity generation, data framing and parallel-to-serial shifting. This block operates on KMIREFCLK, with the incoming KMI clock signal providing the bit rate information. The data is shifted out on the falling edge of the KMICLKIN input detected by the KmiClkInLow input signal.

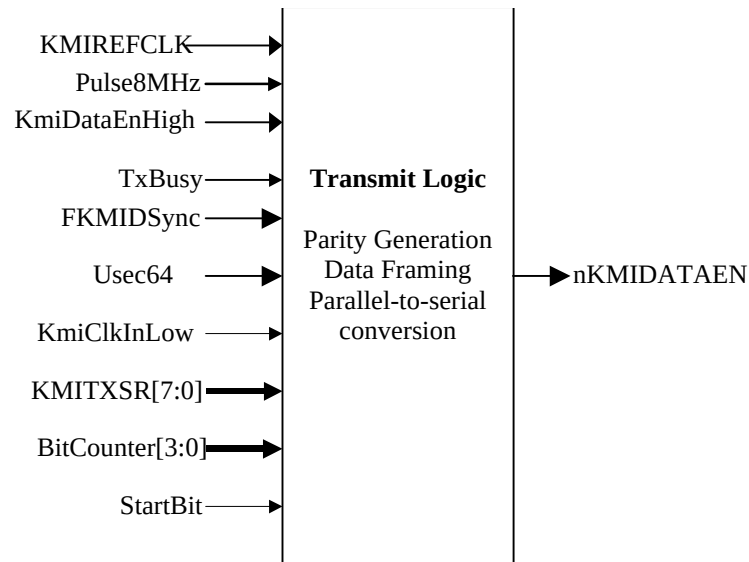


Figure 4.4-6 Transmit Logic

When the controller asserts the TxBusy signal, it indicates to the Transmit section the existence of data to be transmitted. On detecting an asserted TxBusy signal, the data present on the KMITXSR[7:0] bus, which corresponds to the contents of the KMIDATA register, is transmitted. The KMITXSR[7:0] data remains stable for the duration of transmission of the entire data byte. The start bit for transmission is asserted by the controller which is used to drive the nKMIDATAEN signal by the transmit logic as soon as the nKMICLKIN signal is deasserted by the controller. An external Bit counter keeps track of the number of bits transmitted. The data bit in KMITXSR[7:0] pointed to by the current value of this counter is taken as the data bit to be transmitted. The data bit to be transmitted is latched into a D-type clocked by KMIREFCLK one by one in accordance with the bit rate information derived from the incoming KMI clock line. The transmitted data stream is used to drive the nKMIDATAEN pad control signal.

When the synchronised FKMIDSync input signal is asserted by writing to the KMICR register, it forces the data being shifted out to zero. When the controller on a timeout asserts the KmiDataEnHigh signal the nKMIDATAEN signal is pulled high irrespective of the data currently being transmitted.

4.4.5 Receive Logic

The receive logic performs serial-to-parallel conversion on the serial data stream received on the KMIDATAIN input pin and also generates the DataAvl signal to indicate that a valid data has been received. The falling edge on the synchronised and sampled KMICLKIN input signal, detected through the KmiClkInLow input signal is used to sample the KMIDATAIN input line. An external Bit counter keeps track of the number of bits received. The KMIDATAIN input is synchronised to the KMIREFCLK clock domain. The KMIDATAIN input is sampled provided the controller has asserted the RxBusy signal. The stop bit is also detected here.

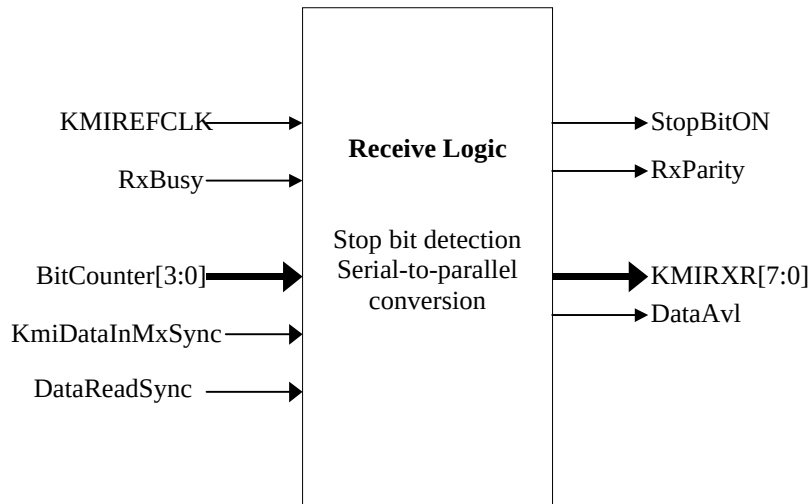


Figure 4.4-7 KMI Receive Logic

When the stop bit is detected and the sampled and synchronised SampKmiClkIn input is high, it asserts the DataAvl signal to indicate that a valid data byte has been received. The DataAvl signal is synchronised to PCLK and the synchronised signal is used to assert the KMIRXINTR interrupt. The DataAvl signal is cleared when the synchronised DataReadSync input signal is asserted, indicating that the received byte has been read. The received parity bit is available as an output from this module.

4.4.6 KMI Controller

The KMI Controller controls the overall operation of the KMI device. It controls the overall transmit and receive operation. If simultaneous request for transmission and reception occur the transmit request is given priority. The KMI controller also controls the start of the external Timer and BitCounter by asserting the ResetTimer and ResetBitCounter signals respectively. These reset signals are disabled only on detecting a valid transmit or receive request. An external timer indicates to the controller the 16 ms timeout duration for transmission and reception as well as the 64 μ s duration for timing the 'request-to-send' during a transmit request. The controller keeps track of the number of bits transmitted or received by decoding an external Bit Counter. The KMI controller logic is enabled on the Pulse8MHz signal. The interface mode for the KMI is indicated by the KMITYPESync signal. In the PS2/AT mode interface the controller waits for an acknowledge pulse at the end of every transmit process. The acknowledge pulse is absent in the 'Legacy' mode interface.

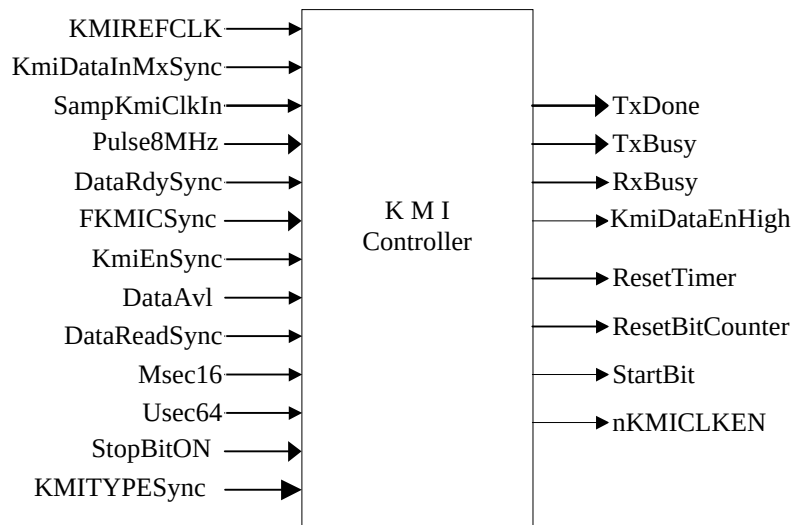


Figure 4.4-8 KMI Controller

When the DataRdydySync signal is asserted it indicates to the controller the existence in the KMIDATA register, of data to be transmitted. On detecting an asserted DataRdydySync signal the KMI controller pulls the KMI Clock line low by enabling the nKMICLKEN pad control signal and asserts the start bit. The clock line is pulled low for 64 μ s prior to transmission to indicate a request-to-send. An external timer indicates to the KMI controller the 64 μ s time by asserting the Usec64 signal. On detecting the Usec64 signal the KMI controller waits for the 8 data bits, one parity and a stop bit to be transmitted. An external Bit Counter indicates to the KMI controller the completion of transmission. When the parity bit is about to be transmitted the controller asserts the TxDone signal to indicate the end of transmission. The TxDone signal is synchronised to PCLK domain and the synchronised TxDoneSync signal is used to clear the DataRdy signal. The interface mode for the KMI is indicated by the KMITYPESync signal programmed by writing to the KMICR register. The controller waits for an acknowledge (ACK) pulse in case of a PS2 /AT-mode interface at the end of transmit process. The acknowledge (ACK) pulse is absent in the Legacy mode. In the WAIT state it waits for the DataRdydySync signal to be deasserted. While in the WAIT state the controller drives the KMI clock line low to inhibit reception and continues to assert the TxDone signal.

If the synchronised and sampled SampKmiClkIn input goes to low it indicates to the KMI controller a request for reception. On detecting the SampKmiClkIn input low the controller checks for the start bit and on detecting a valid start bit the controller proceeds to receive the data and asserts the RxBusy signal. The controller waits for the 8 data bits, one parity and stop bit to be received. An external bit counter indicates to the KMI controller the completion of reception. The controller then inhibits further reception by pulling the KMI clock line low by enabling the nKMICLKEN pad control signal until the received data has been read.

While the inhibit is in progress and DataRdydySync signal is asserted indicating a transmit request the controller repeats the transmit sequence for the next data byte. At the end of the transmission the controller again checks if the received data byte has been read. If the byte has not been read it continues to inhibit further reception.

When during transmission or reception the controller detects the 16ms timeout pulse it aborts the transmit or receive process and waits for the clock line to go idle. The controller forces the data line to idle on a timeout by asserting the KmiDataEnHigh signal.

4.4.7 Timer

This section generates the 64 μ s and 16 ms pulses for the KMI controller. It is a free running timer implemented as three stages clocked by KMIREFCLK. The first stage is implemented as a 6-bit counter, the second stage as a 3-bit counter and the third stage as an 8-bit counter. All the three stages are resettable by the ResetTimer signal generated by the KMI controller. The first stage counter is enabled by the Pulse8MHz signal. The output decode of the first stage goes as an enable for the second stage counter. The output decode of the second stage counter which is the Usec64 pulse goes as an enable for the third stage counter. The decode of the third stage counter generates the Msec16 timeout pulse.

In the test mode the second stage and the third stage counter enables can be bypassed from their normal inputs and enabled by the Pulse8MHz signal. In addition while in test mode the first stage and the third stage counter can be operated in nibble mode.

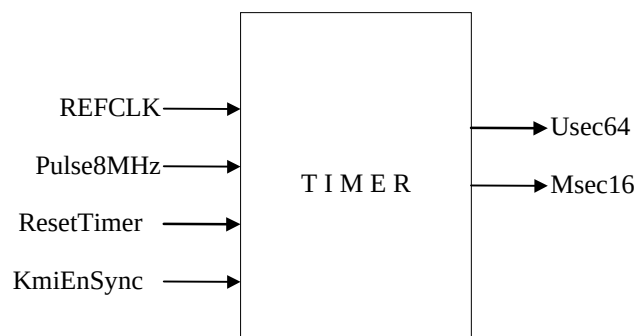


Figure 4.4-9 Timer

4.4.8 Reference Clock Divider

The block generates the Pulse8MHz signal by dividing the KMIREFCLK input. The divider value required is programmed by writing into the KMICLKDIV register. A two buffer synchronisation technique is used to safely load the counter with the programmed value from the KMICLKDIV register. The counter is implemented as a 4-bit down counter clocked by KMIREFCLK and generates the Pulse8MHz signal on reaching zero. This Pulse8MHz signal is used to control the overall operation of the KMI core. By default the Pulse8MHz is always asserted on reset.

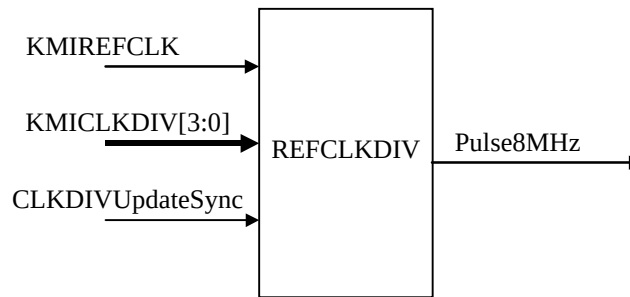


Figure 4.4-10 Reference Clock Divider

4.4.9 KMICLKIN Synchroniser

The KMICLKIN synchroniser synchronises the external KMICLKIN input signal to KMIREFCLK domain. It detects a valid change in the level of the KmiClkInMxSync input by sampling the input line on three successive Pulse8MHz signals to detect a valid KMICLKIN input. If all the three sampled values are the same then the synchronised KMICLKIN input is latched into a D-type to generate the SampKmiClkIn signal. If the three sampled values are not the same then the SampKmiClkIn output signal holds the previous latched value. The falling edge on the incoming KMICLKIN input signal after synchronisation and sampling is indicated by asserting the KmiClkInLow signal.

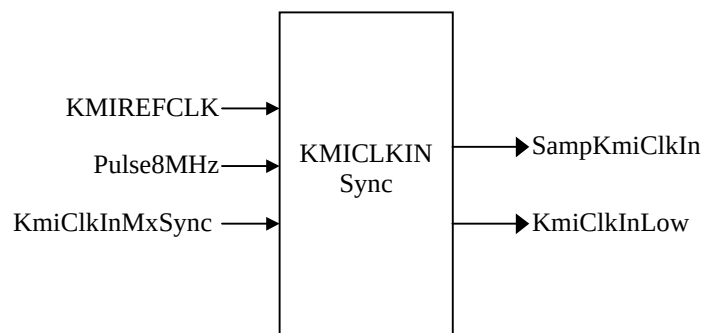


Figure 4.4-11 KMICLKIN Synchroniser

4.4.10 Synchronisers

All synchronisers for signals crossing clock domains have been grouped into two sub-modules – one for signals crossing over from the PCLK domain into the KMIREFCLK domain (SynctoREFCLK) and the other for signals crossing over from the KMIREFCLK domain into the PCLK domain (SynctoPCLK).

The signals crossing clock domains are double-synchronised with flip-flops operating on the rising edge of the clock into which the signals get synchronised.

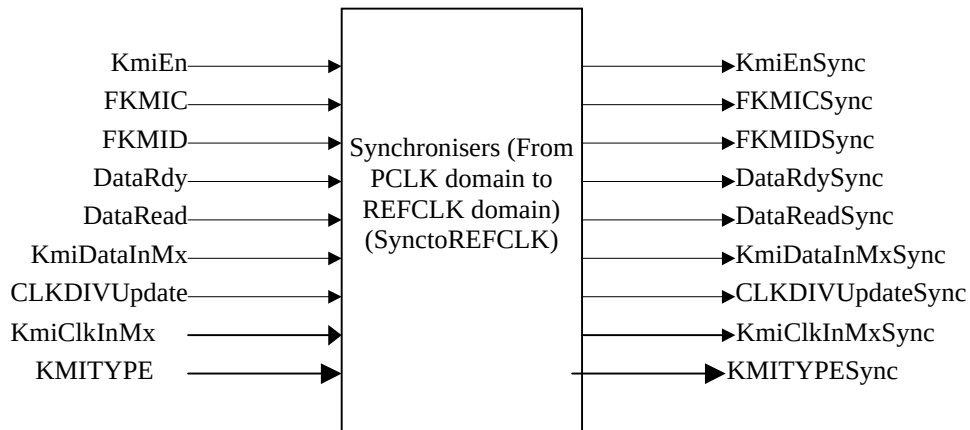


Figure 4.4-12 Synchronisers for signals crossing into KMIREFCLK domain

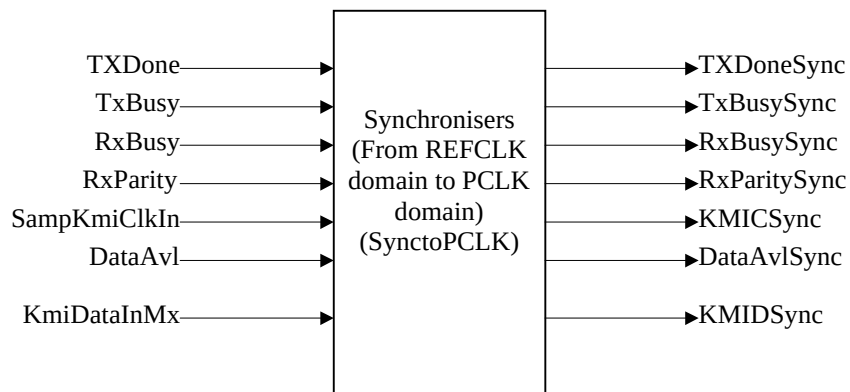


Figure 4.4-13 Synchronisers for signals crossing into PCLK domain

5 KMI IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the KMI is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

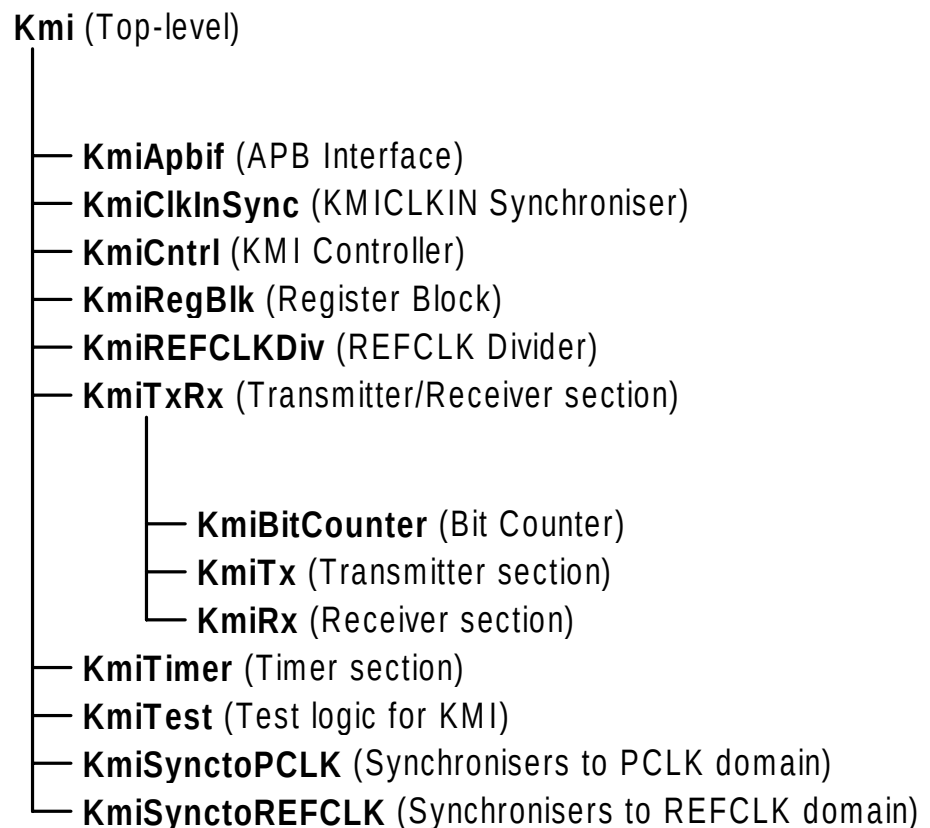


Figure 5.1-1 KMI Design Hierarchy

5.2 Kmi Top Level Module (Kmi)

The top-level module is a structural block which instantiates and interconnects the main functional blocks in the KMI.

5.3 Kmi APB Interface (KmiApbif)

The APB Interface generates decodes for writes into the registers in the address space of the Kmi. It also contains the read interface for the Kmi.

An internal write data bus, PWDatIn[7:0], is generated by gating the write data bus input to the KMI, PWDATA[7:0], with PSEL and PWRITE. This prevents the data bus internal to the KMI from toggling when the device is not being written into or when it is not selected.

5.3.1 Write Interface

The write strobe for each register is generated with a decode of PSEL, PWRITE, PADDRH, PADDRL and PENABLE. **Figure 5.3-1** illustrates the generation of a write strobe for writes into the KMICR[5:0] register and the latching of data into the register. Data is clocked in to KMICR[5:0] on the rising edge of PCLK on which the Write enable (WrenKMICR) is asserted.

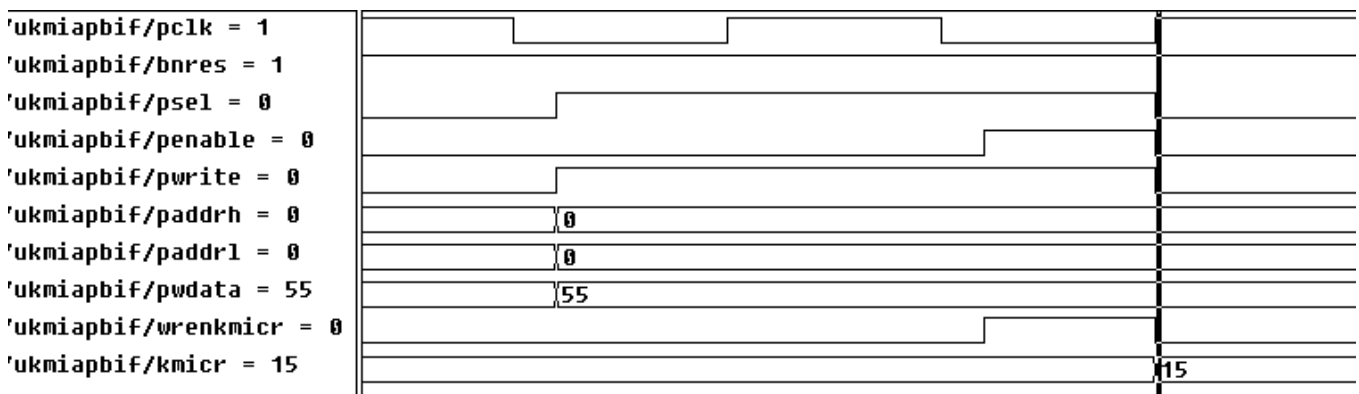


Figure 5.3-1 APB Write Access example

5.3.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the data bus. These flip-flops are clocked by the rising edge of PCLK. Select inputs to the output multiplexer are generated with a combinational decode of PWRITE, PADDRH, PADDRL and PSEL signals. Since these signals are stable from the rising edge of PCLK, almost one full PCLK period is available for the output of the multiplexer to stabilise before data is clocked in on the next rising edge of PCLK. Data is sampled by the APB Bridge on the next rising edge of PCLK on which PENABLE is de-asserted.

The external data bus from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexer bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus when it is not selected. In the KMI, the output multiplexer is left such that, when the device is not selected, it drives zeros onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

Figure 5.3-2 illustrates a read from a KMI register.

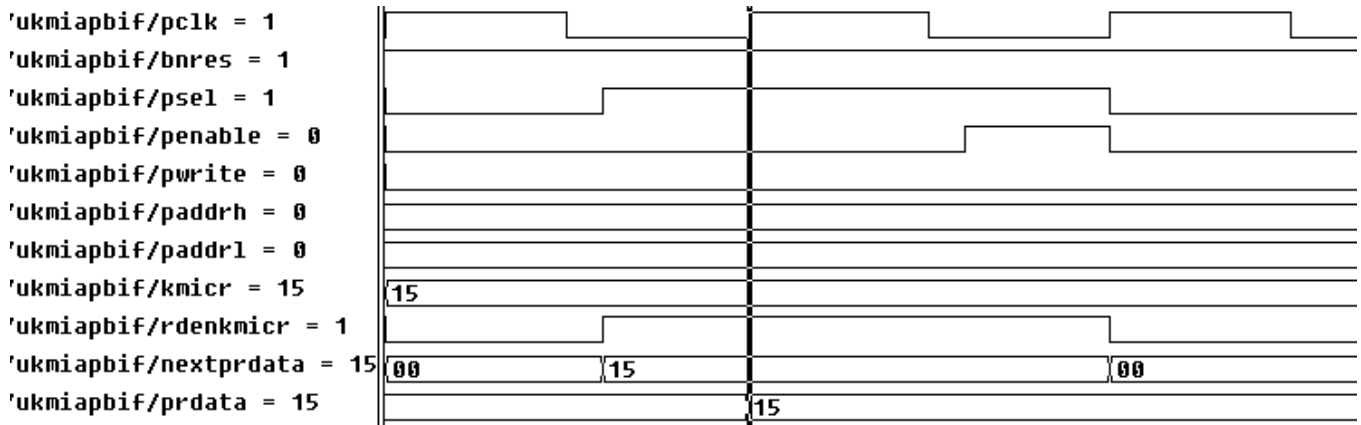


Figure 5.3-2 APB Read Access example

5.4 KMI Reg Block (KmiRegBlk)

This block contains the normal mode registers in the KMI. In addition, it contains the PCLK-domain intermediate buffer for the synchronisation of the KMICLKDIV register to the KMIREFCLK domain. This block also generates the DataRdy signal that is used to indicate to the controller that a byte has been written to the KMIDATA register for transmission. In addition it also generates the KMITXINTR and the KMIRXINTR interrupt. The TxEmpty and RxFull flag are also generated in this block.

5.4.1 KMITXINTR signal generation

Every write to the KMIDATA register results in the clearing of the KMITXINTR provided the interrupt was already set and is enabled. A signal DataRdy is also asserted at the same time. The KMITXINTR interrupt is set when the synchronised TxDoneSync signal and the DataRdy signal are both low. The controller asserts the TxDone signal when it is about to transmit the parity bit. When the TxDoneSync signal is asserted the DataRdy signal is cleared. **Figure 5.4-1** shows waveforms to illustrate the generation of the KMITXINTR signal.

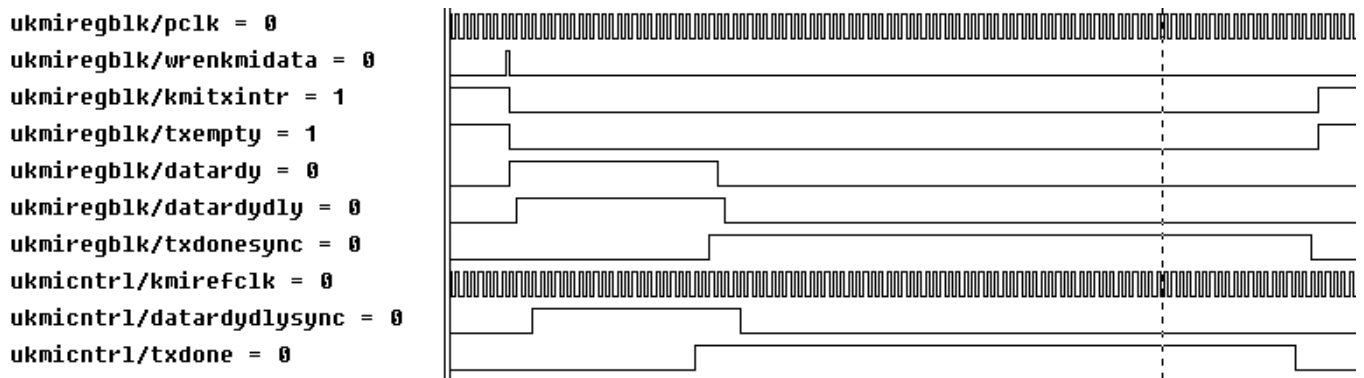


Figure 5.4-1 KMITXINTR generation

5.4.2 KMIRXINTR signal generation

The DataAvl signal is asserted by the receiver logic on receiving a valid byte, the synchronised DataAvlSync signal is used to assert the KMIRXINTR signal. Once this signal is asserted it is cleared on a read of the KMIDATA register. This also results in the generation of the DataRead signal, the synchronised version of which (DataReadSync) is used to clear the DataAvl signal. **Figure 5.4-2** shows waveforms to illustrate the generation of the KMIRXINTR signal. The RxFull signal is also shown below.

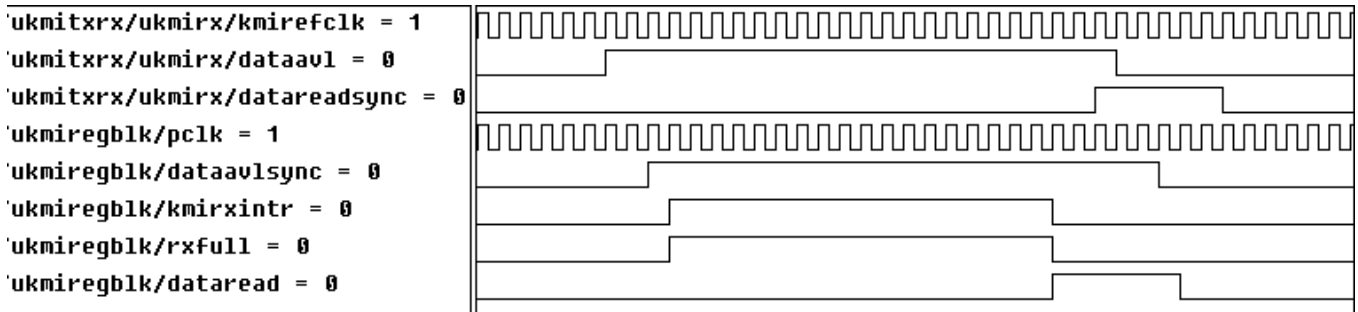


Figure 5.4-2 KMIRXINTR generation

5.4.3 Synchronisation of the KMICLKDIV registers to KMIREFCLK domain

A 2-buffer technique is used to synchronise the contents of the KMICLKDIV register to the KMIREFCLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Block (KmiRegBlock) and the second stage is located in the KMIREFCLK domain in the reference clock divider block (KmiREFCLKDiv).

When there is a write to the KMICLKDIV register, the respective first stage buffer is updated with the new data written. Besides updating the content of the register, the CLKDIVUpdate signal toggles to indicate to the KMIREFCLK domain that the synchronisation process has to start. This signal is double-synchronised to the KMIREFCLK domain and is seen as the CLKDIVUpdateSync signal in the KMIREFCLK domain. A delayed version of this signal is generated by clocking this signal through a D-type clocked by KMIREFCLK. By XOR-ing the CLKDIVUpdateSync signal with its delayed version, a load pulse that is one-KMIREFCLK wide is generated on the HoldCLKDIVWren signal. When the HoldCLKDIVWren signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the KMIREFCLK domain. **Figure 5.4-3** illustrates the waveforms for this synchronisation sequence.

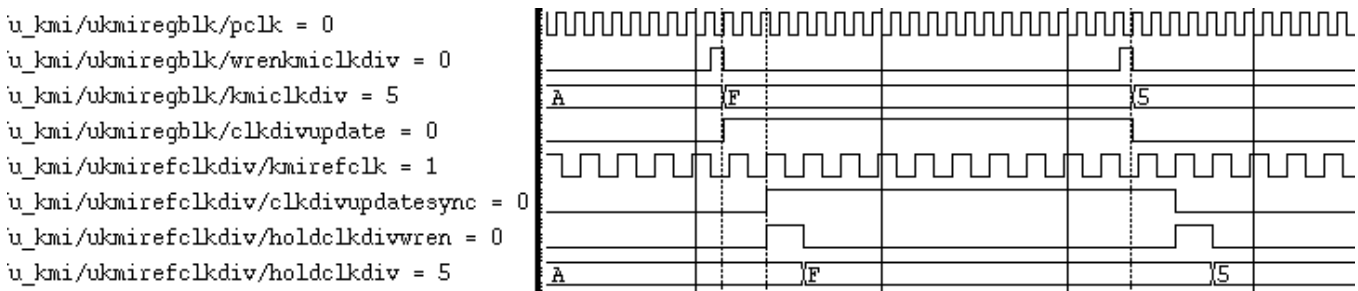


Figure 5.4-3 Waveforms for KMICLKDIV synchronisation

5.5 Generation of the 8MHz pulse (KmiREFCLKDiv)

This module generates the 8MHz pulse by dividing the KMIREFCLK input. The Pulse8MHz signal is a stream of pulses with a pulse width of one KMIREFCLK period and a frequency of 8MHz. The divisor value is programmed by writing to the KMICLKDIV register. As discussed earlier a two buffer synchronisation mechanism is used for transferring the contents of the KMICLKDIV register safely into the KMIREFCLK domain. A 4-bit counter is loaded

with the contents of divisor value in the second stage buffer. The counter then counts down by 1 on every tick of KMIREFCLK and asserts the Pulse8MHz signal on reaching zero. It then reloads the divisor value and the cycle continues. By default on reset the Pulse8MHz signal is always asserted. The waveforms illustrating the generation of the Pulse8MHz signal are shown in **Figure 5.5-1**.

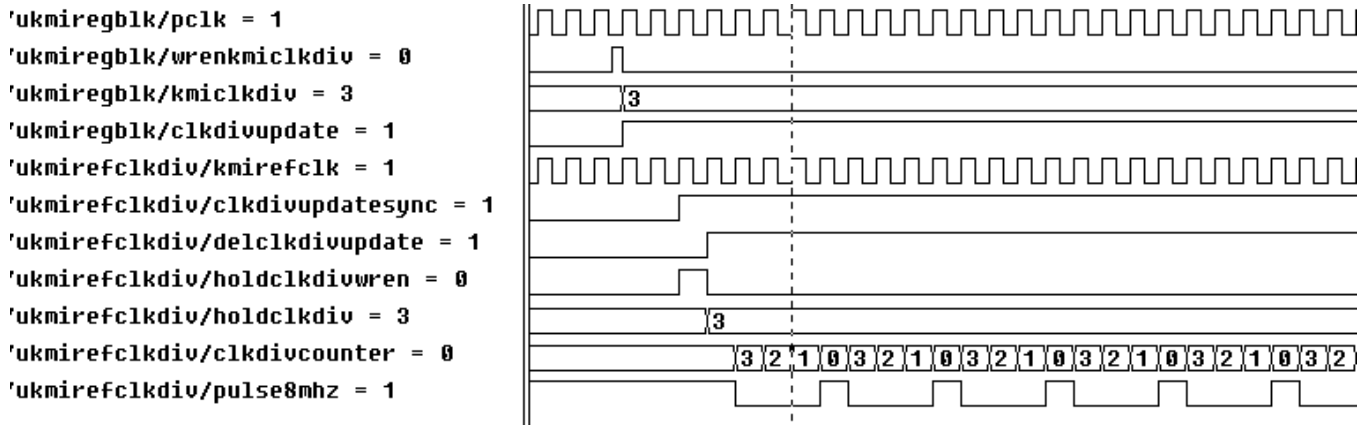


Figure 5.5-1 Generation of the Pulse8MHz signal

5.6 KMICLKIN Synchroniser (KmiClkInSync)

The incoming KMICLKIN signal is synchronised to KMIREFCLK domain. The synchronised signal KmiClkInMxSync is sampled on three successive Pulse8MHz signals to detect a valid KMI clock. If all the three sampled values are the same then the status on the KmiClkInMxSync signal is latched into a D-type to generate the SampKmiClkIn signal. The falling edge on the SampKmiClkIn is indicated by asserting a signal KmiClkInLow that is one Pulse8MHz signal wide. The waveforms illustrating the generation of the SampKmiClkIn and the KmiClkInLow signals are shown in **Figure 5.6-1**.

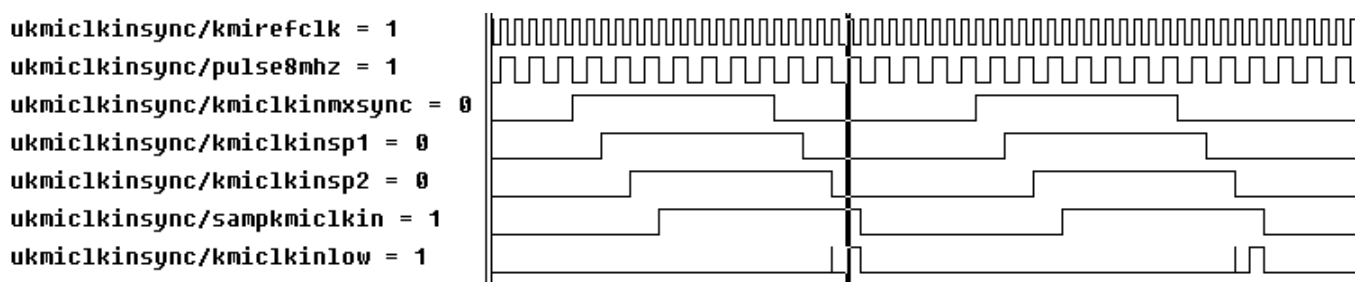


Figure 5.6-1 Generation of SampKmiClkIn and KmiClkInLow

5.7 Transmitter/Receiver logic (KmiTxRx)

This block instantiates the following modules :

- KmiBitCounter : This block keeps track of the number of bits transmitted or received.
- KmiTx : This block implements the transmitter logic for the KMI. It interacts with the KMI controller for the control information and drives the transmit data.

- KmiRx : This block implements the receiver logic for the KMI. It receives the incoming data on the serial data line with the control signals generated by the KMI controller.

5.7.1 Bit Counter (KmiBitCounter)

The bit counter keeps track of the number of bits transmitted or received. The bit counter is implemented as a 4-bit up counter. The bit counter is incremented on every Pulse8MHz pulse on seeing the falling edge of the KMICLKIN line detected by the KmiClkInLow signal. The bit counter is reset at the start of every transmit or receive process by the KMI controller by asserting the ResetBitCounter signal and is enabled only when the nKMICKEN signal is not being driven and the KmiEnSync signal is asserted. The waveforms illustrating the operation of the bit counter are shown in **Figure 5.7-1**.

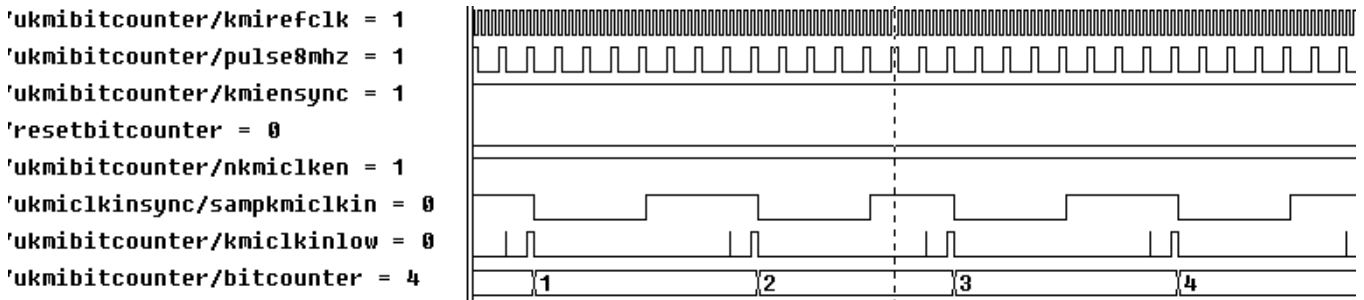


Figure 5.7-1 Waveforms illustrating operation of bit counter

5.7.2 Transmit Section (KmiTx)

The transmit section performs parallel to serial conversion on the transmit data and drives the nKMIDATAEN pad control signal with the bits to be transmitted. It also generates odd parity for the transmit data. The transmit section also drives the nKMICKEN pad control signal at the start of the transmission to indicate a request-to-send.

The KMITXSR[7:0] input to this block is the transmit data from the KMIDATA register. This signal is held stable for the whole duration of transmission and changes are allowed only after the stop bit for a given data byte has been shifted out. The BitCounter[3:0] input specifies the corresponding bit to be shifted out. The transmission occurs only on seeing the TxBusy signal from the controller. The start bit for transmission is generated by the controller at the start of the transmission process. The transmitter drives the start bit at the end of the 64μs time when the request-to-send signal is deasserted. The shifting of data takes place on the falling edge of KMICLKIN input indicated by the KmiClkInLow signal and on seeing the Pulse8MHz signal. The parity bit for the transmitted data is also generated in this block. The waveforms illustrating the transmit process are shown in **Figure 5.7-2**.

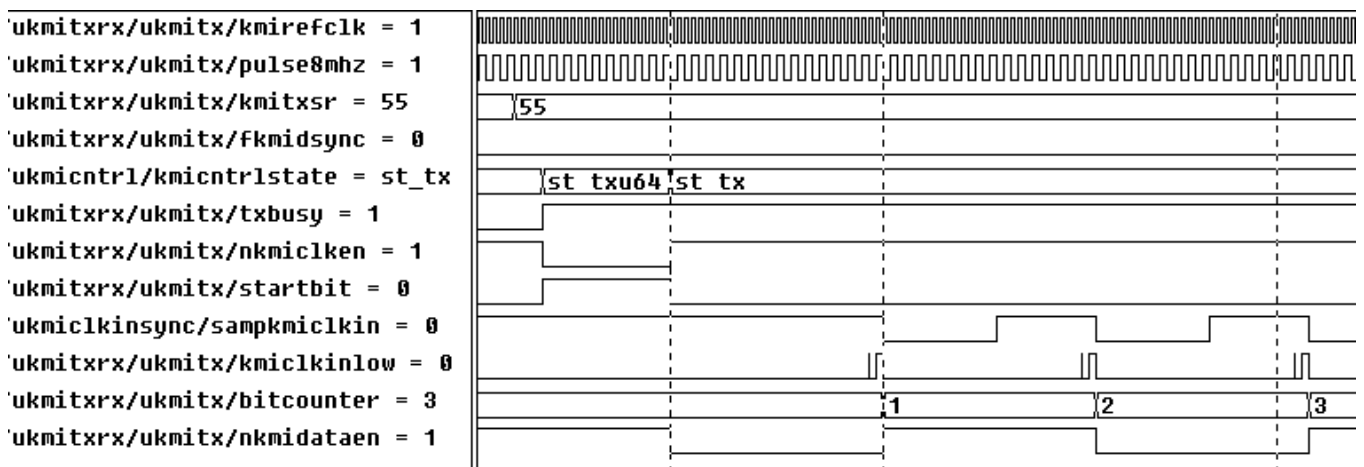


Figure 5.7-2 Transmit operation

5.7.3 Receive Section (KmiRx)

The receive section performs serial to parallel conversion of the received data on the KMIDATAIN line and updates the KMIDATA[7:0] with the received data. It also detects the stop bit at the end of reception and asserts a flag StopBitON on detecting a valid stop bit. The received parity bit is available as a separate output from this block the status of this signal is a readable bit in KMISTAT register which in turn can be read through the APB interface that operates on PCLK. When a data byte has been received correctly it asserts the DataAvl signal, the PCLK synchronised version of which is used to assert the KMIRXINTR signal.

The received data is clocked in on seeing the Pulse8MHz signal and the flag detecting the falling edge on the KMICLKIN line KmiClkInLow signal is also asserted. The received data bit is correctly shifted into the corresponding bit position in the KMIRXR[7:0] register by the bit position pointed to by the bit counter. The waveforms illustrating the reception are shown in **Figure 5.7-3**

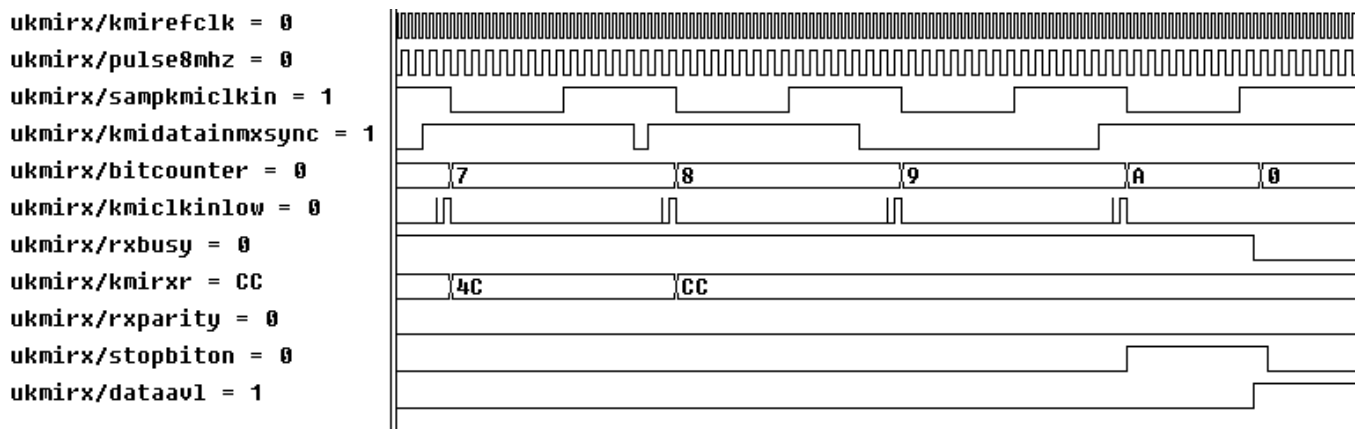


Figure 5.7-3 Receive operation

5.8 KMI Controller state machine (KmiCntrl)

The KMI controller controls the overall transmit and receive operation of the KMI. The states in this machine are :

- ST_RESET. Reset State
- ST_IDLE. State to check for idle (High) state of the clock line.
- ST_TXU64. State to time the request-to-send duration.
- ST_TX. State where the start bit, data bits and the parity bit are shifted out one by one.
- ST_TXDONE. State to indicate transmission done.
- ST_WAIT. State to wait for the transmission done acknowledge from PCLK domain.
- ST_RX. State where the data bits are received during reception.
- ST_LOCK. State to inhibit further reception till received data has been read.
- ST_RECOVER. State to recover from the 16ms timeout during transmission and reception.

The Usec64 signal used in the state machine is used to time the 'request-to-send' signal from the controller by pulling the clock line low by enabling the nKMICLKIN signal (active low) for 64μs after having detected a valid transmit request.

The Msec16 signal used in the state machine is used to detect a timeout during transmission or reception. When this signal is detected the controller aborts the transmit or receive process and enters the ST_RECOVER and waits for the clock line to go Idle.

The KMITYPESync signal indicates to the controller the interface mode (PS2 / AT or Legacy mode) programmed for the KMI.

The state machine depends on the BitCounter[3:0] signal to keep track of the number of bits transmitted or received. The exit from the ST_RX state to ST_LOCK state during reception and the transition from ST_TX state to ST_TXDONE state and ST_TXDONE state to ST_WAIT state during transmission is also dependent on the bit counter. If the FKMICSyn signal is asserted then the nKMICKEN signal is pulled low irrespective of the state the controller is in. In any state when the controller sees the KmiEnSync signal deasserted it returns to ST_RESET state and waits till the KmiEnSync signal is asserted to go to ST_IDLE state. When in the ST_RECOVER state after a timeout the controller forces the data line to high by asserting the KmiDataEnHigh signal.

All transitions between states take place only on seeing the Pulse8MHz signal. The only allowable transitions which are independent of the Pulse8MHz signal are from ST_IDLE state to ST_TXU64 state or ST_LOCK state to ST_TXU64 state on a transmit request and from ST_IDLE state to ST_RX state on a request for reception.

The controller controls the start of the Timer and the BitCounter by deasserting the ResetTimer and ResetBitCounter signals at the start of every transmit or receive request.

Figure 5.8-1 shows a section of the controller state machine. When the nKMIREs signal is asserted the state machine enters the ST_RESET state. When the KmiEnSync signal is asserted and the SampKmiClkIn signal is Idle (High) then the state machine enters the ST_IDLE state. The Timer and the BitCounter are reset.

When the DataRdydlySync signal is asserted, it indicates that transmit data is available in the KMIDATA register. The state machine enters the ST_TXU64 state and deasserts the ResetTimer signal and asserts the TxBusy signal. It also pulls the clock line low by asserting the nKMICKEN signal. When in the ST_IDLE state and the SampKmiClkIn input goes low and the KmiDataInMxSync line is also low the controller enters the ST_RX state to receive the incoming data. It asserts the RxBusy signal to indicate that reception is in progress. If the DataRdydlySync signal and the SampKmiClkIn signal are both detected low simultaneously then the transmit request is given priority.

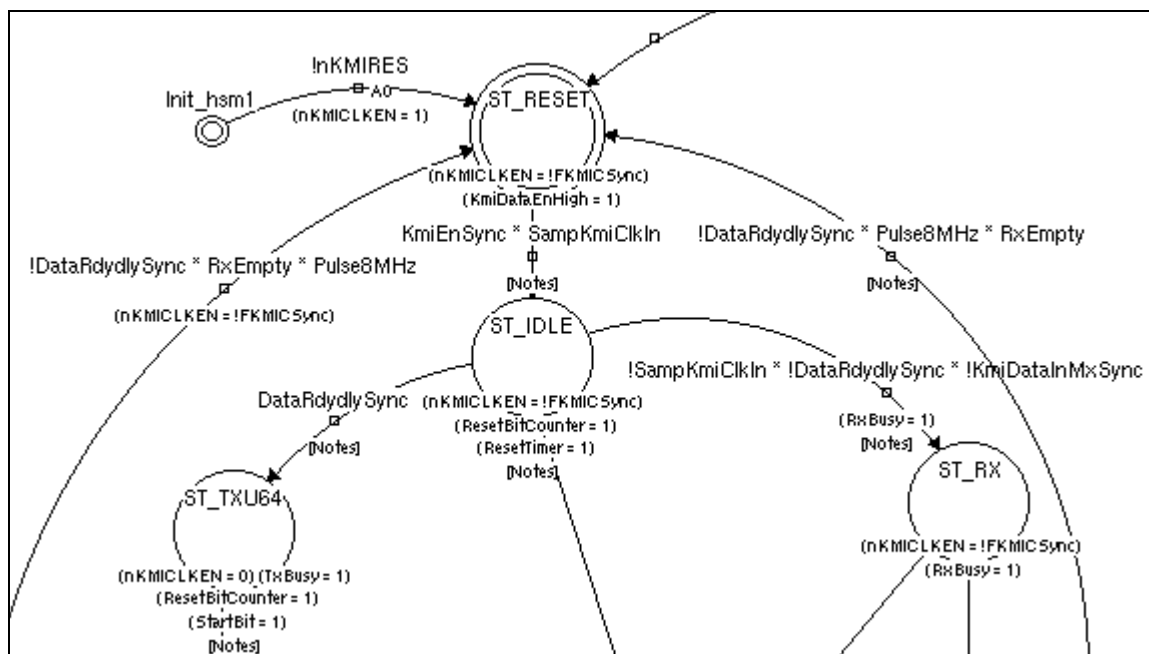


Figure 5.8-1 A section of the state diagram for the controller state machine

Figure 5.8-3 illustrates a section of the state machine during transmission. When the state machine enters the ST_TX state at the end of the 64 μ s duration it continues to assert the TxBusy signal, deasserts the nKMICLK signal and stops driving the start bit. It then waits till the parity bit is about to be shifted and then moves into the ST_TXDONE state and asserts the TxDone signal. If the KMI is programmed for PS2 / AT mode interface then the controller waits for an 'ACK' pulse in response to the transmitted data, and exits the ST_TXDONE state on seeing the SampKmiClkIn line and the KmiDataInMxSync line high. In the legacy mode the controller does not wait for an ACK pulse to exit the ST_TXDONE state. In the ST_WAIT state the controller waits for the DataRdydlySync signal to be deasserted in response to the TxDone signal assertion. While in the ST_WAIT state it inhibits reception by pulling the clock line low by asserting the nKMICLK signal. If on detecting the DataRdydlySync signal being deasserted the received data byte has still not been read then the state machine enters the ST_LOCK state and continues to inhibit further reception till the received data has been read.

When during the transmit process the 16ms timeout happens the state machine aborts the transmit process and enters the ST_RECOVER state and waits for the clock line to go idle. It asserts the KmiDataEnHigh signal so as to pull the data line high.

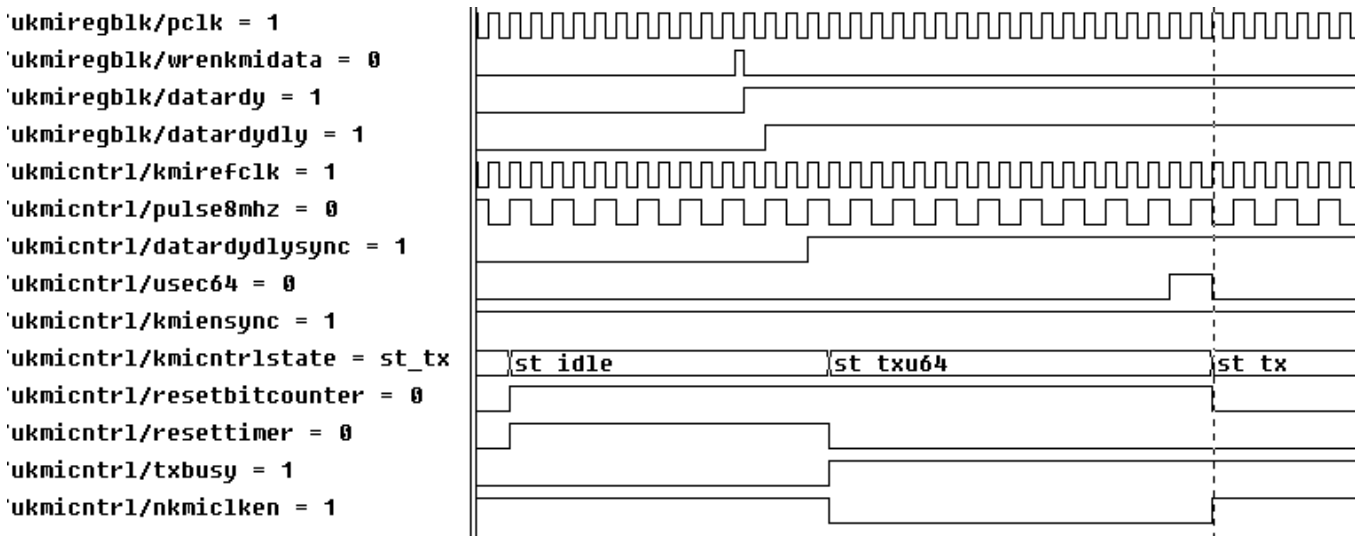


Figure 5.8-2 Generation of Request-to-send

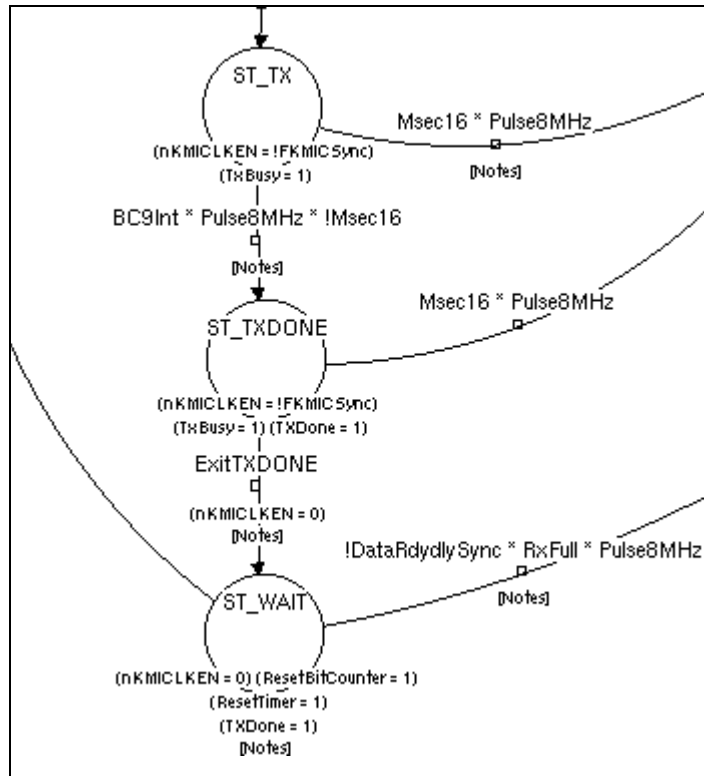


Figure 5.8-3 A section of the state diagram for the controller state machine

Figure 5-8-4 shows a section of the state machine during reception. When the state machine enters the ST_RX state it asserts the RxBusy signal and waits till all the bits including the stop bit have been received. After a valid data byte has been received indicated by the StopBitValid signal being detected and the bit counter having reached ten the state machine enters the ST_LOCK state and inhibits further reception by pulling the clock line low by asserting the nKMICKEN signal. While in the ST_LOCK state if the DataRdydlySync signal is asserted, it indicates that transmit data is available in the KMIDATA register. The state machine moves to ST_TXU64 state to transmit the data.

When in the ST_RX state and a timeout is detected or if a valid stop bit is not detected then the state machine enters the ST_RECOVER state and waits for the clock line to go idle to go back to ST_IDLE state. If it so happens that the state machine had began a transmit process from ST_LOCK state and if a timeout has happened then on coming to ST_RECOVER state the state machine will return to the ST_LOCK state if the received data byte has still not been read else it goes back to ST_RESET state.

The controller exits the ST_LOCK state only on detecting the RxEmpty signal indicating that the received data has been read. The controller can enter the ST_LOCK state on completion of the transmit request if it finds the RxFull signal asserted indicating that the received data has not been read.

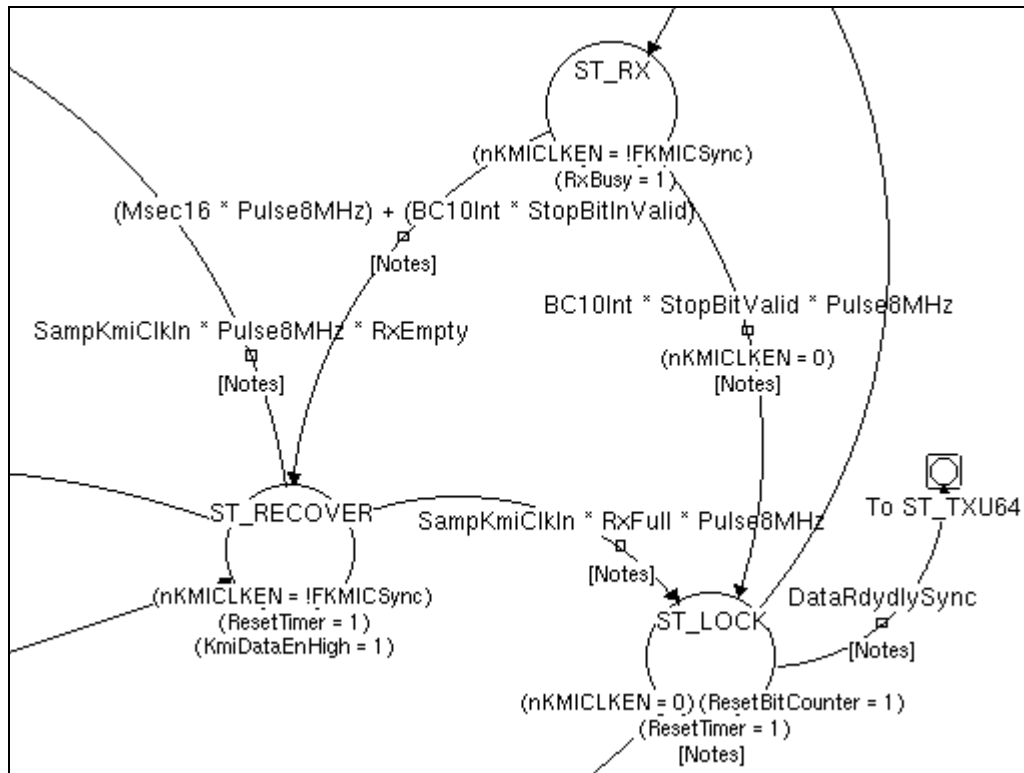


Figure 5.8-4 A section of the state machine when the controller inhibits further reception

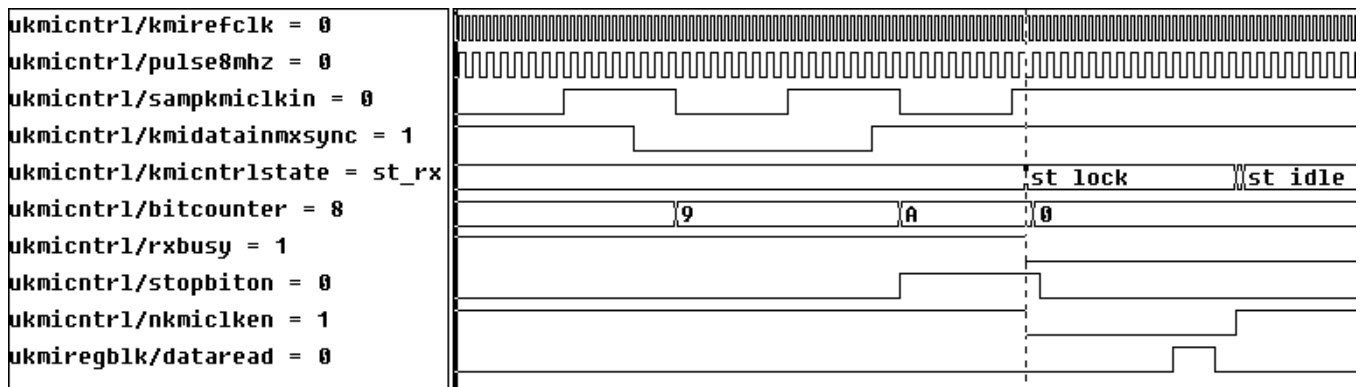


Figure 5.8-5 Waveforms illustrating inhibition on entering LOCK state

5.9 Timer section (KmiTimer)

The timer section generates the 64μs signal for timing the request-to-send signal during the start of transmission and the 16ms signal to indicate a timeout during transmission or reception. The Timer section is implemented as a 17-bit free running counter with three stages. The first stage is a 6-bit counter, the second a 3-bit counter and the third stage an 8-bit counter. The second stage and the third stage counter are enabled by the output decodes of the previous stage while the first stage is enabled by the Pulse8MHz signal. Each stage of the timer counts up by 1 on every KMIREFCLK when enabled. The output decode of the second stage generates the Usec64 signal and that of the third stage timer generates the Msec16 signal, both being one Pulse8MHz wide. All the three stages of

the Timer can be simultaneously reset by the ResetTimer signal from the controller. When the Kmi is not enabled i.e. when the KmiEnSync input signal is not asserted then the timer is disabled.

The first stage and the second stage counters are reloaded on overflow. The third stage is reloaded with zero on reaching 0xF9 in normal mode. The first stage and the third stage of the timer can be enabled in nibble mode for test purposes. When the bypass for the second stage and the third stage are asserted the enable for these stages are bypassed and are driven by the Pulse8MHz signal directly. These two features help in reducing the number of vectors required for testing the timer and the timeout during the transmit and receive process.

The waveforms illustrating the generation of the Usec64 and the Msec16 are shown in **Figure 5.9-1**

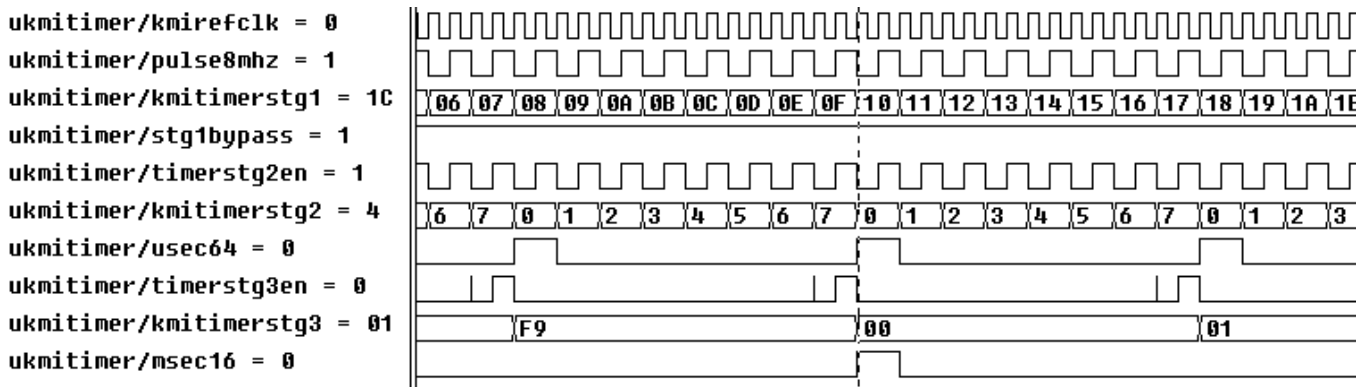


Figure 5.9-1 Generation of the 64µs and 16ms timeout pulses

5.10 Synchronisers to KMIREFCLK domain (KmiSynctoKMIREFCLK)

This block contains all synchronisers in the design that synchronise signals entering the KMIREFCLK domain.

The DataRdydy signal is from the KmiRegBlk operating on PCLK. It is required for the controller section to detect if transmit data is available. The DataRead signal is from the KmiRegBlk operating on PCLK. It is required by the controller to detect if the received data has been read. The CLKDIVUpdate signal is from the PCLK domain part of the control logic required to synchronise the contents of the KMICLKDIV register to the KMIREFCLK domain. This signal is required by the KmiREFCLKDiv logic, which contains the KMIREFCLK-domain part of the control logic required to synchronise the contents of this register to the KMIREFCLK domain. The FKID signal is from the register block and is required to force the nKMIDATAEN pad control signal to low when asserted, irrespective of the data being transmitted or not. The FKMIC signal is also from the register block and forces the nKMICLKEN pad control signal to low irrespective of the state of the controller. The KmiEn signal enables/disables the KMI transmission/reception. The KMITYPE signal indicates the interface mode 'PS2 / AT' or 'legacy' mode for the KMI and is required by the controller section.

Asynchronous input signals such as KMICLKIN and the KMIDATAIN are also synchronised to the KMIREFCLK domain.

5.11 Synchronisers to PCLK domain (KmiSynctoPCLK)

This block contains all synchronisers for signals entering the PCLK domain.

The TxDone signal is from the controller section. It is required by the register block to deassert the DataRdy signal on completion of transmission and to assert the KMITXINTR signal. The TxBusy and the RxBusy signals are from the controller. These signals are readable bits in the KMISTAT register through the APB interface. The DataAvl signal is from the Receive section. It is required by the register block to detect when a received data byte is available so as to assert the KMIRXINTR interrupt. The RxParity signal is also from the Receive section. The status of this signal is a readable bit in the KMISTAT register through the APB interface.

Asynchronous input signals such as KMICLKIN and KMIDATAIN, are also synchronised to PCLK domain. These signals need to be synchronised to PCLK because they are readable bits in the KMISTAT register through the APB interface.

5.12 Test logic (KmiTest)

The Test block performs the following functions:

- Generation of Test clock enable (ClkEn) generation based on the contents of KMITCR.
- Generation of Test resets (nReset, nKMIREs) based on the contents of KMITCR and the level on the SCANMODE pin.
- Write interface for the Test register
- Test input stimulus muxing for non-AMBA inputs (KMICLKIN and KMIDATAIN)

In the normal mode, the ClkEn signal is pulled high. If the TestModeEn bit is set, the device is said to be in the test mode. In the test mode, if the TestClkEn bit is not set, then ClkEn is pulled high. Else, the internal test clock enable is selected based on the RegClkMode bit value. If RegClkMode is set, then a pulse on the ClkEn line is generated only on accesses to the KMITCER register address i.e. when the SelKMITCER input is asserted. If the RegClkMode bit is not set, then a clock enable is generated on every access to the device. The ClkEn signal is generated with PENABLE and PSEL as gating terms.

When the SCANMODE pin is low, the test reset generated by OR-ing the external reset with the test reset is driven as the internal reset (nReset for PCLK-domain logic and nKMIREs for KMIREFCLK-domain logic). When SCANMODE is high, the test reset does not have any effect and the external resets BnRES and nKMIREs are driven on the internal reset lines. The Test registers themselves are not affected by test reset.

Non-AMBA inputs such as KMICLKIN and KMIDATAIN are muxed with the corresponding bits in the KMITISR register when the TESTINPSEL bit in the KMITCR register is set.

6 KMI TEST DETAILS

6.1 KMI Testbenches

6.1.1 VHDL Bus Compliance Testbenches

The ARM PrimeCell KMI VHDL test world uses two variants of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbenches are designed to be used with most common VHDL simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbenches are “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.1-1 illustrates the VHDL testbench for running production test vectors.

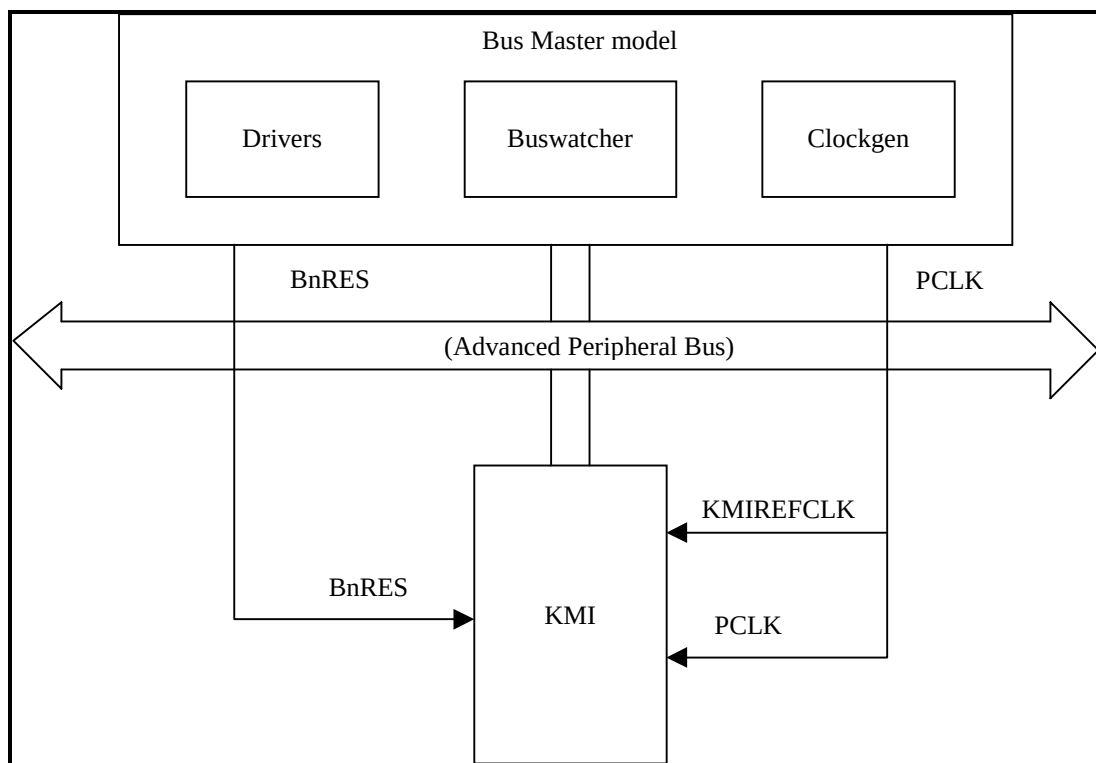


Figure 6.1-1 VHDL testbench for running production test vectors

Figure 6.1-2 illustrates the hierarchy for the VHDL production testbench.

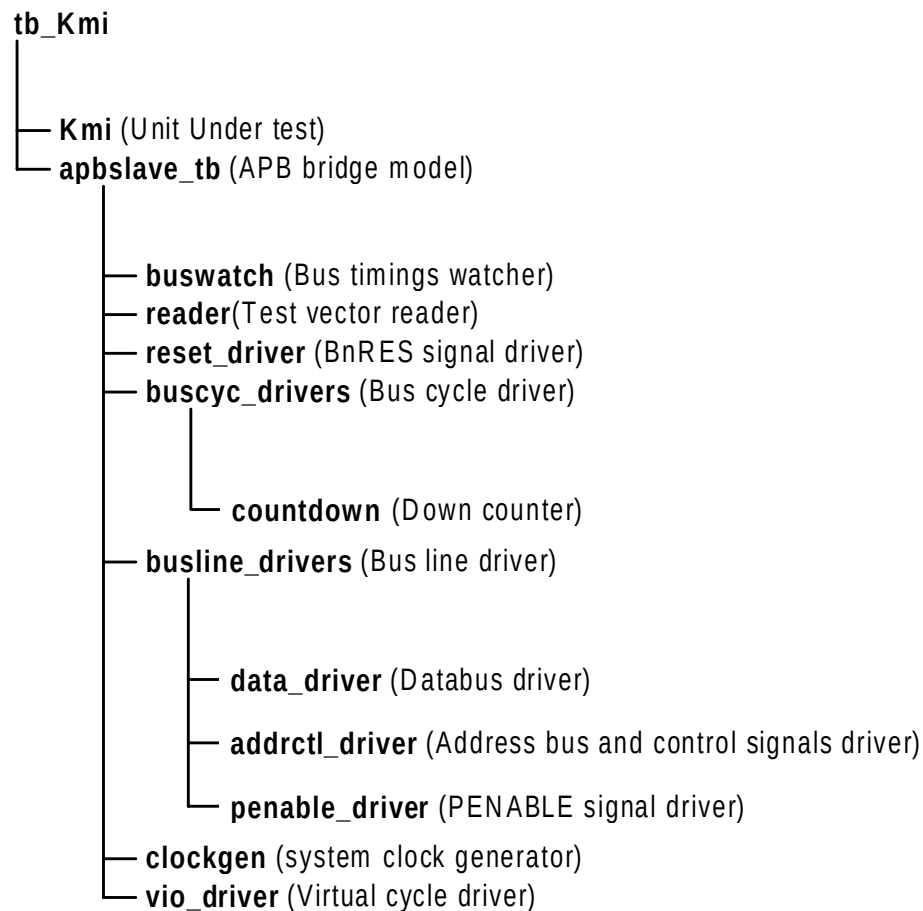


Figure 6.1-2 Testbench hierarchy for production vectors

The testbench, **tb_Kmi.vhd** is used for production level testing using internal clock enabling mechanisms. This block instantiates the KMI (Unit Under Test) and the behavioural model for the APB bridge. The key point about this testbench is that both PCLK and KMIREFCLK are fed from the system Bus clock. Since production test vectors use clock enabling to control internal device activity, these two clocks need to run at the same frequency to keep test code frequency independent.

The SCANMODE pin is tied low to keep it from interfering with the test process.

The default test frequency is set at 50MHz. This is exactly the same frequency to which the KMI was constrained during synthesis. PCLK and KMIREFCLK are set to the BCLK frequency defined in **tb_Kmi.vhd** using the **tclkh** and **tckl** generic values when instantiating **apbslave_tb.vhd**.

Figure 6.1-4 illustrates the hierarchy for the free-running clocks VHDL testbench.

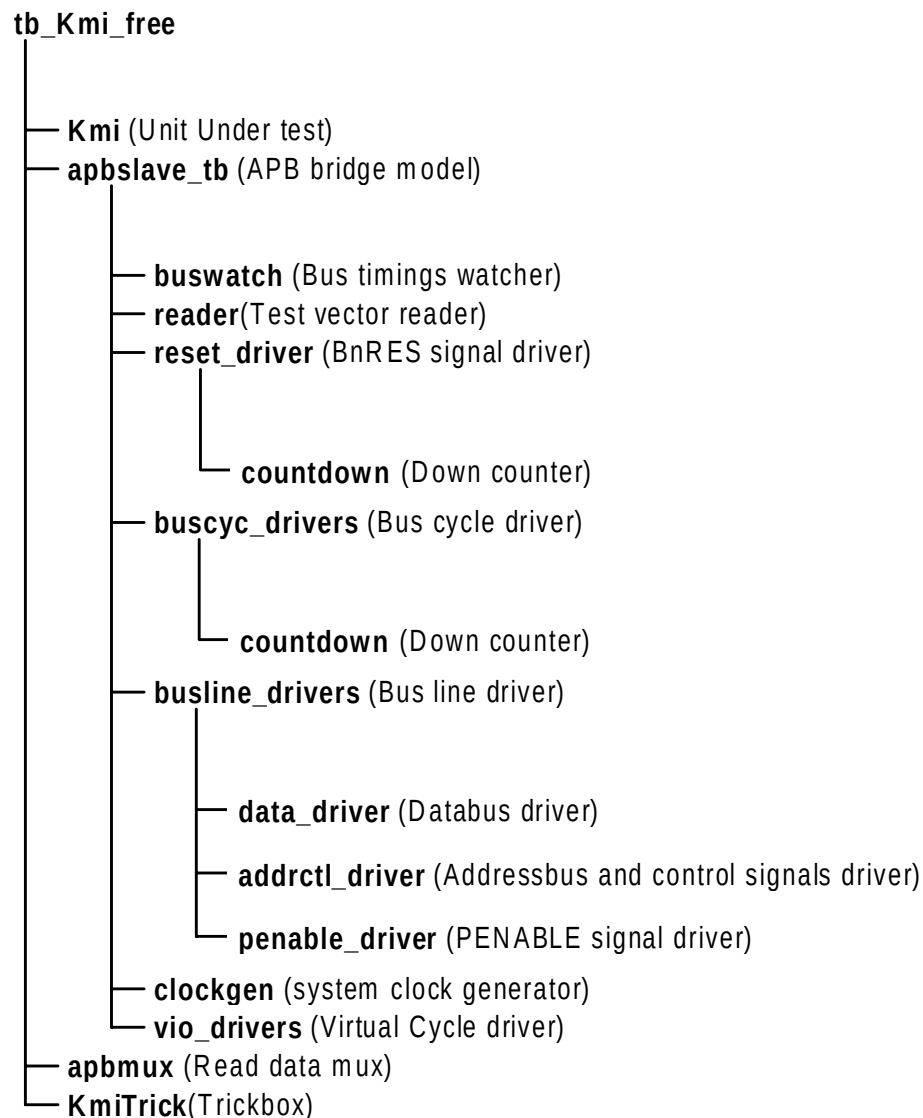


Figure 6.1-4 VHDL Testbench hierarchy for free-running vectors

The testbench, **tb_Kmi_free.vhd** is used for functional simulations of the KMI. This block instantiates the KMI (Unit Under Test), the behavioural model of the APB Bridge, the Trickbox and the APB Read multiplexer. Unlike in the case of **tb_Kmi.vhd**, PCLK and KMIREFCLK need not be the same frequency. However, they are set to 50MHz since this was the synthesis constraint. The KMIREFCLK is generated by the Trickbox to facilitate frequency modifications from within the free running test code.

The PCLK and KMIREFCLK clock period value can be re-defined in the free running test program **Kmi_free.c**. PCLK is also set to the BCLK frequency defined in **tb_Kmi_free.vhd** using the **tclkh** and **tckl** generic values when instantiating **apbslave_tb.vhd**.

All the non-AMBA inputs to the KMI are driven by the Trickbox.

6.1.1.1 Unit Under Test

The Unit Under Test may be one of the following

- Kmi.vhd VHDL RTL
- Kmi_net.vhd VHDL netlist

One out of these two versions was referenced via a Model Technology Inc, ModelSim library mapping file, "libmap" in the **BusTalk/vhdl/modeltech** directory.

6.1.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB Bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the infile.bif file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the BnRES signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

6.1.1.3 Trickbox

The functionality of the KMI is verified via a Trickbox model.

6.1.1.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.vhd file in the **BusTalk/vhdl/tbench** directory. The buswatch block which is in the **Bustalk/vhdl/buswatcher** directory checks the Read/Write Data bus timings and reports any violations in the timings.

6.1.1.5 APB Multiplexer

The read data buses from the KMI and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

6.1.1.6 Vector Sets

The KMI has two sets of vectors for validating functionality

Free Running Tests

The **Kmi_free.c** file in the **BusTalk/bustests** directory exercises the functionality of the KMI with all clocks free-running. This test depends on the Trickbox to verify proper device operation. The test vectors are generated as a BusTalk Input File (*.bif) which is read by the VHDL free running testbench.

Clock Enabled Production Tests

The **Kmi_prod.c** file in the **BusTalk/bustests** directory exercises the functionality of the KMI in clock enabled mode. These vectors should be used for final production level testing on the tester. The test vectors are generated as a BusTalk Input File (*.bif) which is read by the VHDL production testbench.

6.1.1.7 Generics

The KMI testbench provides some configurability options using generics. These generics are configurable through the tb_Kmi.vhd or the tb_Kmi_free.vhd files.

XonPSEL

XonPSEL is used in the addrctl_driver.vhd file that resides in **BusTalk/vhdl/bid**.

This generic is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.vhd, the reset_driver.vhd and the buscyc_drivers.vhd which reside in **BusTalk/vhdl/bid**.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.vhd which resides in **BusTalk/vhdl/bid**.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd which resides in **BusTalk/vhdl/buswatcher**.

This generic is used to suppress the checking of the PRDATA timings when BnRES signal is asserted. If **SuppressOnReset** is TRUE, the PRDATA set-up and hold timing violations are not flagged when BnRES is asserted. If **SuppressOnReset** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of BnRES.

Mesg_enb

Mesg_enb is used in the buswatch.vhd which resides in **BusTalk/vhdl/buswatcher**.

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when

the UUT is not selected. This generic will have to be set to FALSE in testbenches which include more than one slave.

Data_check

Data_check is used in the apbmux.vhd which resides in **BusTalk/vhdl/tbench**.

This constant is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **Data_check** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This constant is used in testbenches which include more than one slave.

6.1.1.8 Running Simulations

The **README** file in the **BusTalk/vhdl/tbench** directory gives step by step instructions for running simulations using the clock-enabled or the free-running tests on the VHDL source code.

Toggle coverage numbers when the production test vectors are run on the VHDL in both RTL and netlist forms are available in the above mentioned directory:

Kmi.prod.bif.rtl.vhdl.untog

Kmi.prod.bif.net.vhdl.max.untog

Kmi.prod.bif.net.vhdl.min.untog

A sample .untog file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	C/T = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the vhdl or verilog in rtl or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the **BusTalk/vhdl/tbench/logs** directory. The log files and the toggle reports above were created using the Model Technology Inc ModelSim simulator.

Kmi.prod.bif.rtl.vhdl.log

Kmi.prod.bif.net.vhdl.max.log

Kmi.prod.bif.net.vhdl.min.log

Kmi.free.bif.rtl.vhdl.log

Kmi.free.bif.net.vhdl.max.log

Kmi.free.bif.net.vhdl.min.log

6.1.2 Verilog Bus Compliance Testbenches

The ARM PrimeCell KMI Verilog test world uses two variants of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test benches are designed to be used with most common Verilog simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test benches are “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.1-5 illustrates the Verilog testbench for running production test vectors.

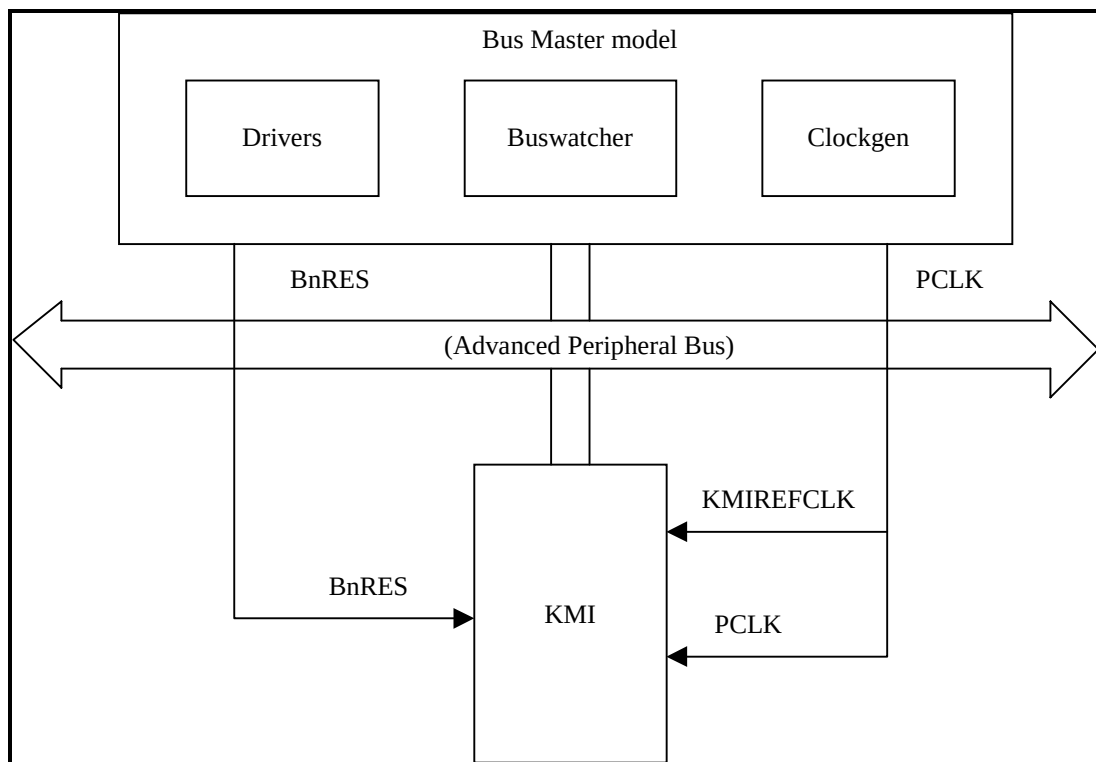


Figure 6.1-5 Verilog Testbench for running production test vectors

Figure 6.1-6 illustrates the hierarchy for the Verilog production testbench.

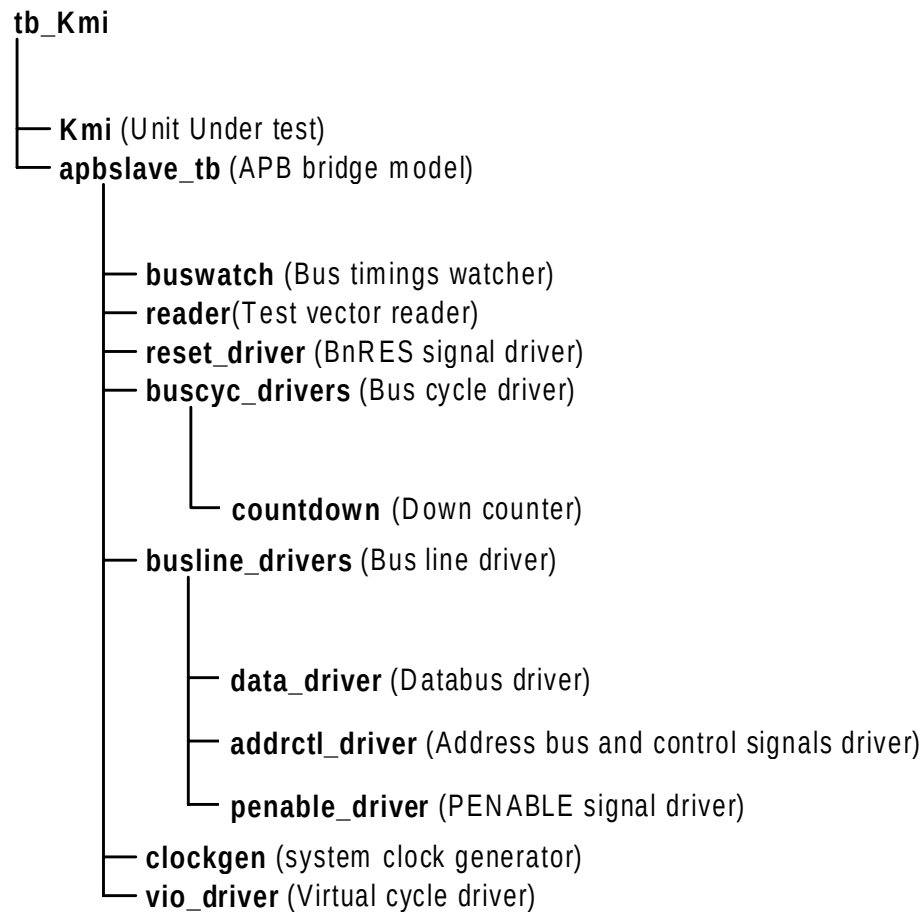


Figure 6.1-6 Verilog Testbench hierarchy for production vectors

The testbench, **tb_Kmi.v** is used for production level testing using internal clock enabling mechanisms. This block instantiates the Kmi (Unit Under Test) and the behavioural model for the APB bridge. The key point about this testbench is that both PCLK and KMIREFCLK are fed from the system Bus clock. Since production test vectors use clock enabling to control internal device activity, these two clocks need to run at the same frequency to keep test code frequency independent.

The SCANMODE pin is tied low to keep it from interfering with the test process. The Modem status lines are held at their inactive levels.

The default test frequency is set at 50 MHz. This is exactly the same frequency to which the KMI was constrained during synthesis. PCLK and KMIREFCLK are set to the BCLK frequency defined in timing.v using the Tclkh and Tckl 'define values when instantiating apbslave_tb.v in tb_Kmi.v. Note, that the 'define values override the local parameters defined in clockgen.v in the common/ directory.

Figure 6.1-7 illustrates the Verilog testbench for tests using free-running clocks.

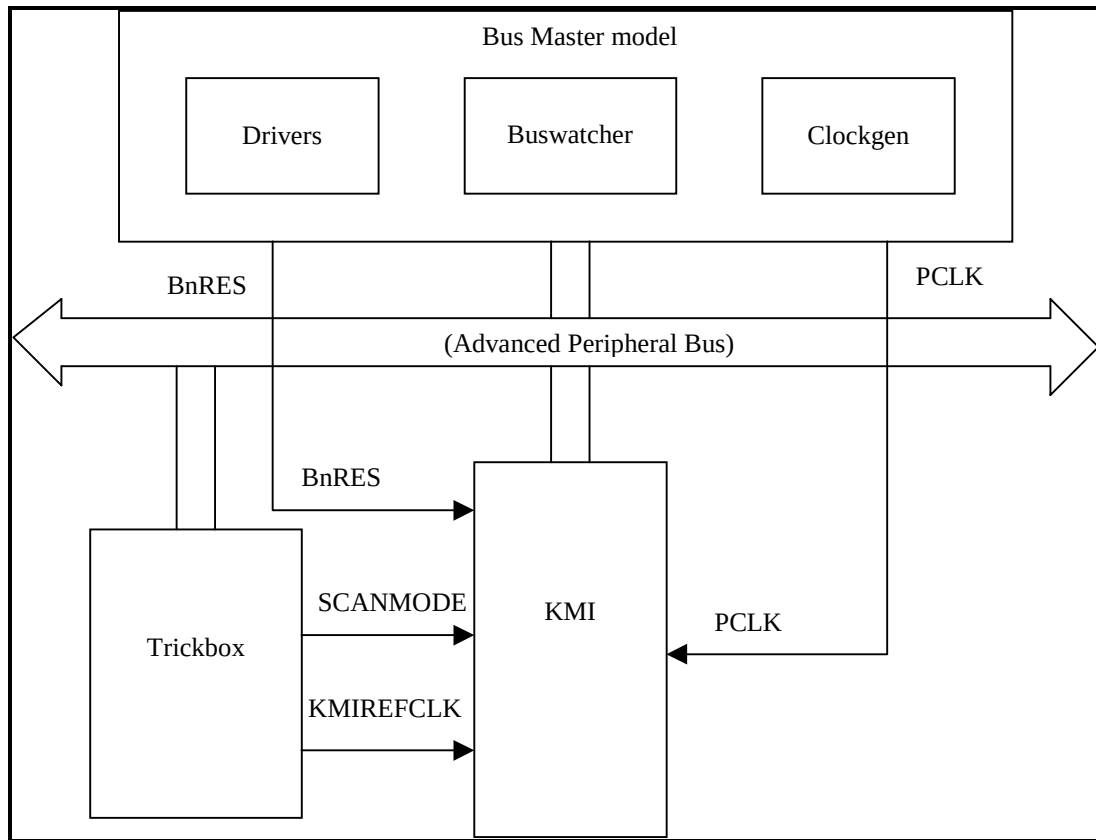


Figure 6.1-7 Verilog Testbench for running test vectors using free-running clocks

Figure 6.1-8 illustrates the hierarchy for the free-running clocks Verilog testbench.

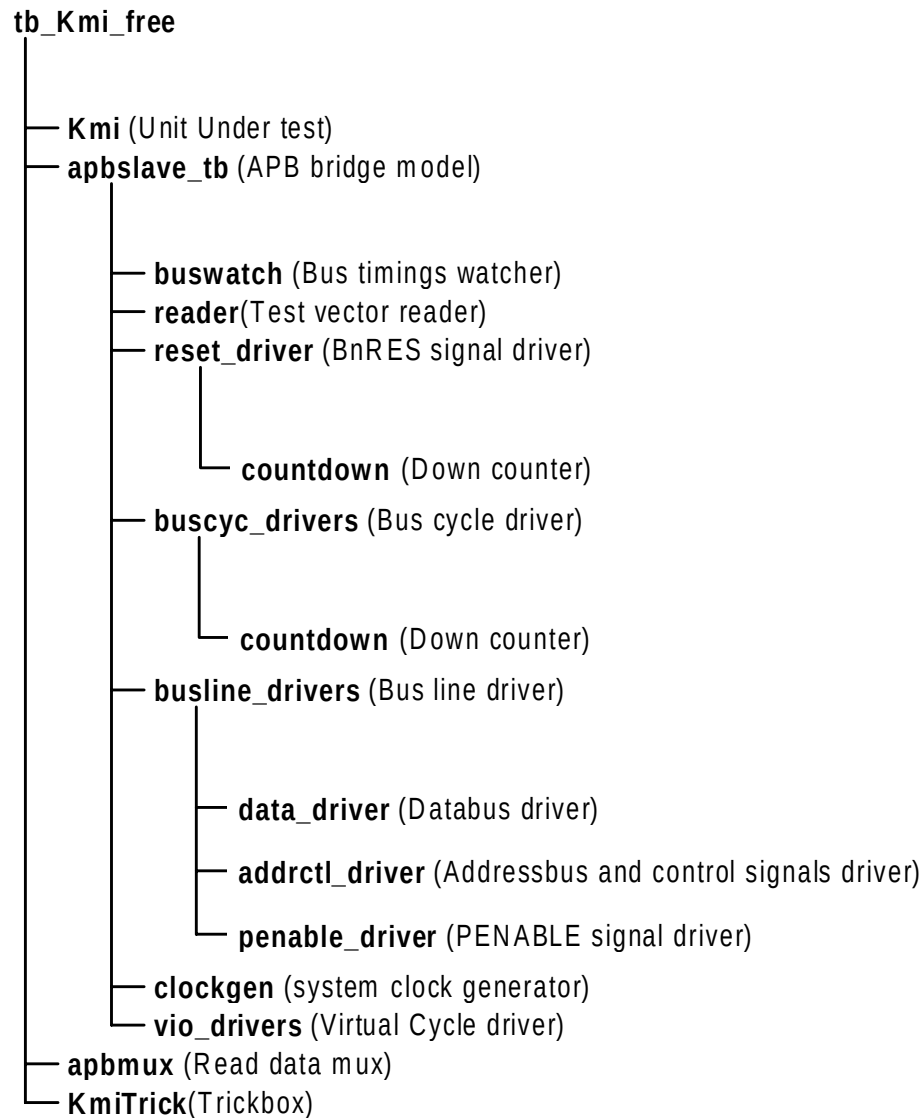


Figure 6.1-8 Verilog Testbench hierarchy for free-running vectors

The testbench, **tb_Kmi_free.v** is used for functional simulations of the KMI. This block instantiates the Kmi (Unit Under Test), the behavioural model of the APB Bridge, the Trickbox and the APB Read mux. Unlike in the case of **tb_Kmi.v**, PCLK and KMIREFCLK need not be the same frequency. However, they are set to 50 MHz since this was the synthesis constraint. The KMIREFCLK is generated by the Trickbox to facilitate frequency modifications from within the free running test code.

The PCLK and KMIREFCLK clock period value can be re-defined in the free running test program Kmi_free.c. PCLK is also set to the BCLK frequency defined in timing.v using the Tclkh and Tckl 'define values when instantiating apbslave_tb.v in tb_Kmi_free.v. Note, that the 'define values override the local parameters defined in clockgen.v in the common/ directory.

All the non-AMBA inputs to the KMI are driven by the Trickbox.

6.1.2.1 Unit Under Test

The Unit Under Test may be one of the following

- Kmi.v Verilog RTL
- Kmi_net.v Verilog netlist

One out of these two versions may be referenced via the library mapping file “libmap” in the **BusTalk/vlog/modeltech** directory.

6.1.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the infile.bif file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the BnRES signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

6.1.2.3 Trickbox

The functionality of the KMI is verified via a Trickbox model.

6.1.2.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.v file in the **BusTalk/vlog/tbench** directory. The buswatch block which is in the **Bustalk/vlog/buswatcher** directory checks the Read/Write data bus timings and reports any violations in the timings.

6.1.2.5 APB Multiplexer

The read data buses from the KMI and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

6.1.2.6 Vector Sets

The KMI has two sets of vectors for validating functionality

Free Running Tests

The **Kmi_free.c** file in the **BusTalk/bustests** directory exercises the functionality of the KMI with all clocks free-running. This test depends on the Trickbox to verify proper device operation. The test vectors are generated as a BusTalk Input File (but the Verilog version is named *.sim as opposed to the VHDL *.bif version) which is read by the Verilog free running testbench.

Clock Enabled Production Tests

The **Kmi_prod.c** file in the **BusTalk/bustests** directory exercises the functionality of the KMI in clock enabled mode. These vectors should be used for final production level testing on the device (IC) tester. The test vectors are generated as a BusTalk Input File (but the Verilog version is named *.sim as opposed to the VHDL *.bif version) which is read by the Verilog production testbench.

6.1.2.7 Parameters

The KMI testbench provides some configurability options via parameters and defines. All the parameters, except MESG_ENB and SUPPRESSONRESET are configurable through the **tb_Kmi.v** or the **tb_Kmi_free.v** files. MES_ENB and SUPPRESSONRESET are configurable through the buswatch.v that resides in **BusTalk/vlog/buswatcher**. The define, DATA_CHECK is configurable through the apbmux.v that resides in **BusTalk/vlog/tbench**.

XonPSEL

XonPSEL is used in the **tb_Kmi.v** and **tb_Kmi_free.v** files that resides in **BusTalk/vlog/tbench**.

This define is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.v, the reset_driver.v and the buscyc_drivers.v which reside in **BusTalk/vlog/bid**.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.v which resides in **BusTalk/vlog/bid**.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESSONRESET is used in the buswatch.v which resides in **BusTalk/vlog/buswatcher**.

This parameter is used to suppress the checking of the PRDATA timings when BnRES signal is asserted. If **SUPPRESSONRESET** is **1**, the PRDATA set-up and hold timing violations are not flagged when BnRES is asserted. If **SUPPRESSONRESET** is set to **0**, PRDATA set-up and hold timing violations are flagged irrespective of BnRES.

MESG_ENB

MESG_ENB is used in the buswatch.v which resides in *BusTalk/vlog/buswatcher*.

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to **0**. If **MESG_ENB** is set to **1**, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to **0** in testbenches which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the apbmux.v which resides in *BusTalk/vlog/tbench*.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **DATA_CHECK** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.1.2.8 Running Simulations

The **README** file in the *BusTalk/vlog/tbench* directory gives step by step instructions for running simulations using the clock-enabled or the free-running tests on the Verilog source code.

Toggle coverage numbers when the production test vectors are run on the Verilog in both RTL and netlist forms are available in the *BusTalk/vlog/tbench/logs* directory.

Kmi.prof.bif.rtl.verilog.vsim.untog

Kmi.prod.bif.net.verilog.max.vsim.untog

Kmi.prod.bif.net.verilog.min.vsim.untog

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	C/T = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the Verilog in RTL or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the **BusTalk/vlog/tbench/logs** directory. The log files below, and the toggle reports above, were created using the Model Technology Inc ModelSim simulator.

Kmi.prod.bif.rtl.verilog.vsim.log

Kmi.prod.bif.net.verilog.max.vsim.log

Kmi.prod.bif.net.verilog.min.vsim.log

Kmi.free.bif.rtl.verilog.vsim.log

Kmi.free.bif.net.verilog.max.vsim.log

Kmi.free.bif.net.verilog.min.vsim.log

Also contained in this directory are simulation logs created using the Cadence Verilog-XL simulator:

Kmi.prod.bif.rtl.verilog.vxl.log

Kmi.prod.bif.net.verilog.max.vxl.log

Kmi.prod.bif.net.verilog.min.vxl.log

Kmi.free.bif.rtl.verilog.vxl.log

Kmi.free.bif.net.verilog.max.vxl.log

Kmi.free.bif.net.verilog.min.vxl.log

6.1.3 Unification of BusTalk and TicTalk

The KMI testbench can automatically produce **.tif** formatted vectors from BusTalk source in addition to the regular **.bif** formatted vectors.

The facility to generate vectors of either format is provided by the addition of **ifdef** control to the **busmacros.c** file in the **BusTalk/bustests** directory of the world. If the **TIF #define** is left uncommented in the **config.h** file in the **bustests** directory, **busmacros.c** will produce **.tif** formatted vectors during compile time. If the **#define** is left commented (default) , **.bif** formatted vectors are generated. The following sections detail how each APB Slave Compliance command is interpreted if the **TIF #define** is set.

TestStart()

By default, the **TestStart()** command inserts a pre-determined number of Idle cycles on the APB bus. However, in the Tic world, the startup delays are determined by the testbench itself and not by user written commands. In fact, Tic commands take effect only after BnRES is deactivated. Hence, the busmacros simply prints a comment in the **.tif** file, on encountering a **TestStart()** command in the BusTalk source.

TestEnd()

By default, the **TestEnd()** command appears as the string **"TE"** in the **.bif** file.

If the TIF #define is set, the **TestEnd()** command is replaced by an **A-A-E** sequence.

PNW()

The **PNW()** command does not have a direct equivalent in the TicTalk world since any access to an APB Slave device address space will cause the APB Bridge to assert the corresponding PSEL along with PSTB. A comment is printed into the **.tif** file for each **PNW()** command.

PSW()

The **PSW()** command appears as an **A-A-W** sequence in the **.tif** file.

PNR()

Like the **PNW()** command, the **PNR()** command does not have a direct equivalent in the TicTalk world. A comment is printed into the **.tif** file for each occurrence of the **PNR()** command.

PSR()

The **PSR()** command appears as an **A-A-R-A** sequence in the **.tif** file. Note that the read is followed by an Address-Only cycle for bus-turnaround.

PI()

The **PI()** command is implemented using an **A-A-L** sequence in the **.tif** file.

RES()

Since the assertion of BnRES is not under command control in the TicTalk world, the **RES()** command simply prints out a comment in the **.tif** file. BnRES assertion occurs automatically under testbench control at the start of the simulation.

PO()

Note that the POLL command does not have any equivalent in the TicTalk world. Hence, BusTalk source code to be converted into **.tif** formatted vectors should not include any **PO()** commands.

6.1.4 Vector Generation

A **make sourcefilename** (without the **.c** extension) in the **bustests** directory will create **.bif** formatted and **.sim** formatted vectors in the **bustests/invec** directory. This is the default setting.

To create **.tif** formatted vectors from the same source, the following two changes need to be made.

the **#define TIF** needs to be set in the **config.h** file in the **bustests** directory.

setenv VECT_TYPE TIF at the unix command line.

Now **make sourcefilename** (without the **.c** extension) will create **.tif** formatted vectors in the **bustests/invec** directory.

To revert back to **.bif** generation, do the following

/* #define TIF */ in **config.h**

unsetenv VECT_TYPE

6.1.5 AMBA TICTalk testbenches

In addition to the BusTalk testbenches, TICTalk VHDL and Verilog testbenches are supplied to apply TICTalk production test vectors to the KMI. The TICTalk testbenches is based on a subset of components from the ARM Example AMBA System (EASY). For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

The TICTalk production test vectors used with these testbenches can be applied to the KMI when it part of an AMBA system design.

Figure 6.1-9 illustrates the testbench to apply TICTalk test vectors.

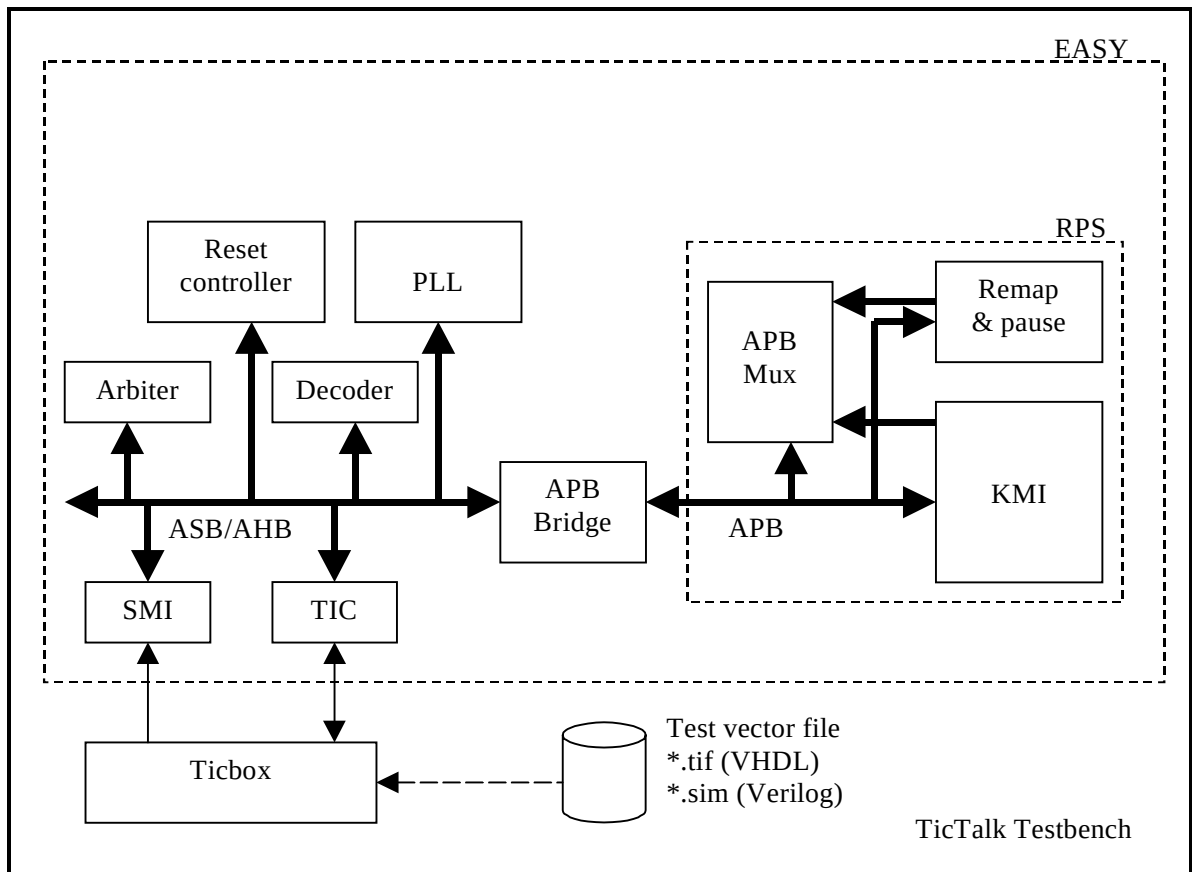


Figure 6.1-9 Testbench for running TICTalk production test vectors

Figure 6.1-10 illustrates the hierarchy for the TicTalk testbench.

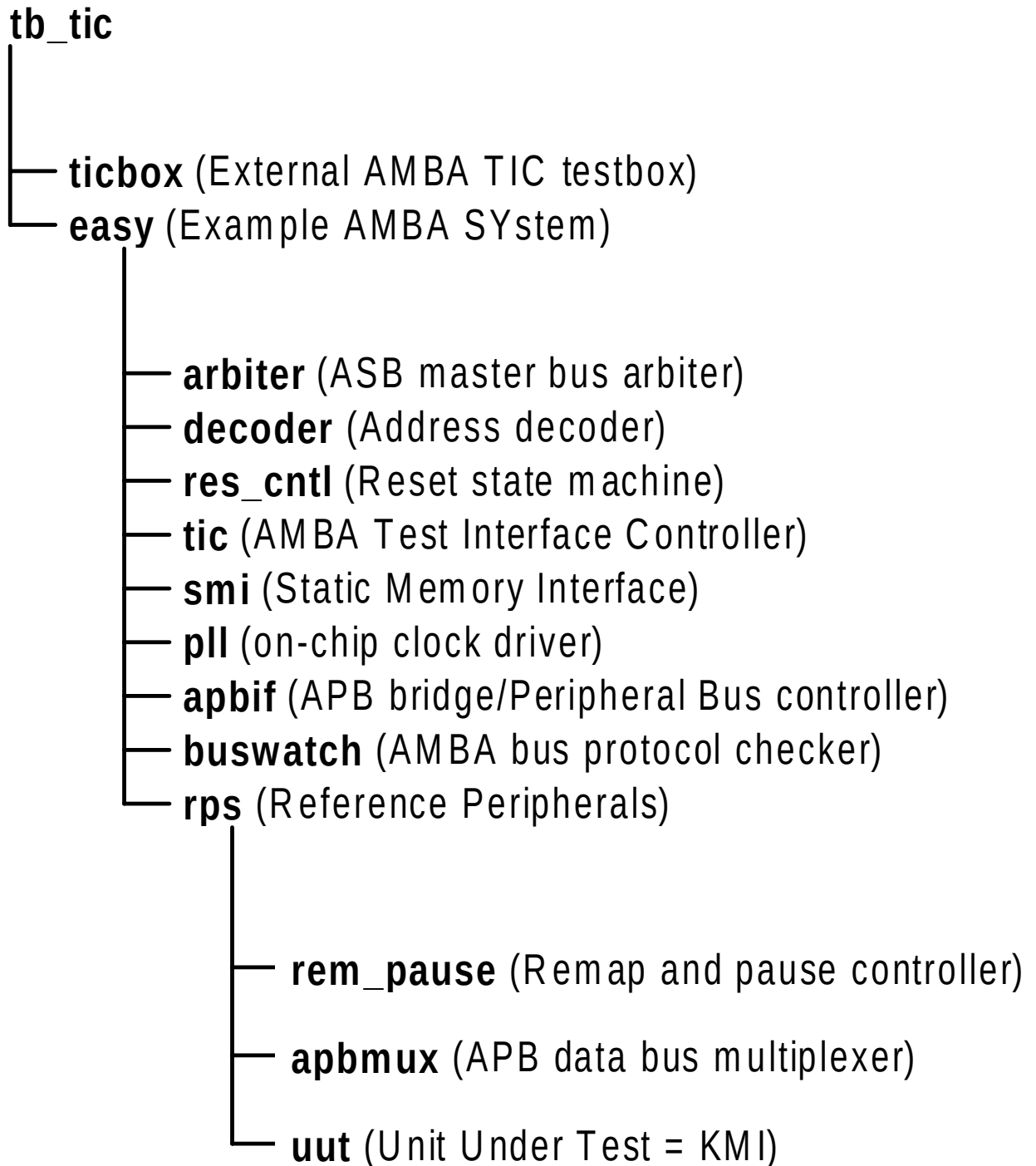


Figure 6.1-10 Testbench hierarchy for-running TicTalk test vectors

6.2 KMI Trickbox

The Trickbox is a programmable AMBA APB device designed to provide a test engineer a means to drive the non-APB input pins of the KMI and sample the non-APB responses of the same.

The Trickbox reads the data transmission line of the KMI and performs serial to parallel conversion of data received on it. The Trickbox is capable of converting parallel data to serial data and driving the serial data on the data reception lines of the KMI. The Trickbox also has the width measurement and the error checking capabilities.

The following sections detail the functionality of the Trickbox.

6.2.1 Trickbox Signal Description

AMBA APB signal descriptions

Name	Type	Source/Destination	Description
PCLK	Input	From Test Bench	APB clock
BnRES	Input	From Test Bench	Bus reset signal, Active Low
PADDR [7:2]	Input	From Test Bench	Subset of APB address bus
PRDATA[15:0]	Output	To Test Bench	Subset of unidirectional APB read data bus
PWDATA[15:0]	Input	From Test Bench	Subset of unidirectional APB write data bus
PSELT	Input	From Test Bench	Trickbox select signal from decoder. When set to 1 this signal indicates the slave device is selected and that a data transfer is required.
PENABLE	Input	From Test Bench	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	Input	From Test Bench	APB transfer direction signal, indicates a write access when HIGH, read access when LOW.

KMI Data signals

Name	Type	Source/Destination	Description
KDATAIN	Input	From KMI	KMI received serial data input
KDATAOUT	Output	To KMI	KMI transmitted serial data output, defaults to the marking state 1, when reset.

KMI Interrupt lines

Name	Type	Source/Destination	Description
KMIRXINTR	Input	From KMI	KMI Receive interrupt (active HIGH).
KMITXINTR	Input	From KMI	KMI Transmit interrupt (active HIGH).
KMIINTR	Input	From KMI	KMI Interrupt Line (active HIGH).

KMI Clock lines

Name	Type	Source/Destination	Description
KMICLKOUT	Output	To KCLK pad	KMI Clock Output
KMICLKIN	Input	From KCLK pad	KMI Clock Input

KMI Reference Clock and Reset

Name	Type	Source/Destination	Description
KMIREFCLK	Output	To KMI	KMI Reference Clock.
NKMIRST	Output	To KMI	KMI Reset line, Active Low.

Miscellaneous

Name	Type	Source/Destination	Description
SCANMODE	Output	To KMI	Scan Mode Enable Output

6.2.2 Trickbox functional description

The architectural behaviour of the Trickbox is similar to that of the KMI itself. The Trickbox contains the following major modules.

APB Interface and the Register Block

The APB is a local secondary bus which provides a low-power extension to the higher AHB (or ASB) within the AMBA system hierarchy. The APB interface generates read and write decodes for accesses to status/control registers and transmit/receive registers. The register block stores data written or to be read across the APB interface. All registers are READ / WRITE, although the writeability feature of some registers is not used. Such registers are specified explicitly.

Trickbox register summary

Address (Offset from TrickBox Base)	Type	Width	Reset Value	Name	Description
0x00	R/W	8	0x00	KmiTrRXREG/ KmiTrTXREG	Data written to the Transmit register and data read from the Receive register
0x04	R/W	5	0x00	KmiTrCnREG	KMI Control Register
0x08	R/W	7	0x20	KmiTrSTAT	Reads from this Register return Status Information
0x0c	R/W	9	0x00	KmiTrCLKL	KCLK LOW time Value Register
0x10	R/W	9	0x00	KmiTrCLKH	KCLK HIGH time Value Register
0x14	R/W	16	0x00	KmiTrDSI	Setup on KDATA to KCLK Falling for Receive
0x18	R/W	16	0x00	KmiTrDHI	Setup on KDATA to KCLK Rising for Receive
0x1c	R/W	16	0x00	KmiTrDSO	Setup on KDATA to KCLK Rising for Transmitting
0x20	R/W	16	0x00	KmiTrDHO	Setup on KDATA to KCLK Falling for Transmitting
0x24	R/W	8	0x00	KmiTrREFCLK	KMI Reference Period
0x28	R/W	16	0x00	KmiTrRG	KCLK LOW from controller to KCLK LOW from peripheral for request to send
0x2c	R	6	0x13	KmiTrFFLAG	Fifo Status Flags
0x30	R	5	0x17	KmiTrCHECKPIN	TrickBox reads the programmed pin
0x34	R/W	5	0x00	KmiTrTIMOUT	Time Out Generation Register
0x38	R/W	8	0x00	KmiTrCLKDIV	Trickbox Reference Clock divisor
0x3c	R/W	4	0x00	KmiTrMODEREG	TrickBox Clock/Reset Mode Register
0x40	R/W	2	0x00	KmiTrTIMESTAT	TrickBox Timing Error Status

Transmit and Receive Registers

The transmit register is 8-bits wide. Any data written to the Trickbox data register is stored in this register till it is read out by the transmit logic.

The receive register is 8-bits wide. Received data is stored in the receive register by the receiver logic until read out by the CPU across the APB interface. Associated Error bits are being stored in the Status Register.

Transmitter Logic

The transmitter logic performs parallel-to-serial conversion on the data read from the transmit register. Control logic outputs the serial bit stream beginning with a start bit, data bits, LSB (Least Significant Bit) first, followed by parity bit and then stop bit.

Receiver Logic

The receiver logic performs serial-to-parallel conversion on the received bit stream after a valid start pulse has been detected. And in the PS2_AT mode it also sends the ACK pulse at the end of transmission. The Trickbox needs to detect the high to low edge of the start bit to correctly interpret the data bit stream. Parity and framing error checking are also performed and the received data is written to the receive register.

6.2.3 Trickbox Register Description

The base address of the Trickbox is not fixed and may be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

KmiTrRXREG/KmiTrTXREG: Data register [8] (+0x00)

This is an 8-bit read/write register for all data transfers to or from the internal Trickbox. This write initiates transmission from the Trickbox if the Trickbox is enabled. If the Trickbox is disabled, the data is held in this register till the Trickbox is enabled.

On reads, this register gives the 8-bit data byte received by the Trickbox, and the KMISTAT register holds five status bits associated with that data character. The value read from KmiTrRXREG is zero on reset .

Bits	Name	Read/Write	Function
7:0	KmiTrRXREG/ KmiTrTXREG	R/ W	Receive (read) data character Transmit (write) data character

KmiTrCnREG: Control Register [5] (+0x04)

All the bits are cleared on reset.

Bits	Name	Read/Write	Function
4	SCANMODE	R/W	This bit is transparent to the SCANMODE Pin.
3	LEGACYBIT	R/W	Setting this bit to 1 enables the Legacy mode test.
2	KDATA Width Measurement bit	R/W	If this bit is set then the Trickbox internally measures the time between two edges of KDATA and sets the KDATA_WIDTH_ERR to 1 if the time does not match the expected bit width
1	Forced Parity Error Bit	R/W	This bit introduces a parity error in the data frame.
0	Trickbox Enable	R/W	Trickbox is enabled if this bit is set to 1.

KmiTrSTAT: KMI Status Register [7] (+0x08)

Bits	Name	Read/Write	Function
6	KmiTrREFCLKOn	R/W	Reference Clock Enable Status
5	KmiTrPCLKOn	R/W	PCLK Clock Enable Status
4	KmiTrDWIDTHERR	R/W	This bit is set if an error is detected in KDATA width measurement.
3	KmiTrTXBUSY	R/W	The Trickbox is transmitting data.
2	KmiTrRXBUSY	R/W	The Trickbox is receiving data.
1	KmiTrFRAMEERR	R/W	This bit is set if framing error is detected .
0	KmiTrPARITYERR	R/W	This bit is set If parity error is detected in the received data.

KmiTrCLKL: Trickbox Clock Low Time Register [9] (+0x0c)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
8:0	KmiTrCLKL	R/W	KMI Clock Low time .

KmiTrCLKH: Trickbox Clock High Time Register [9] (+0x10)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
8:0	KmiTrCLKH	R/W	KMI Clock High time

KmiTrDSI: Trickbox DSI Register [16] (+0x14)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
15:0	KmiTrDSI	R/W	Setup on KDATA to KCLK falling for Receive

KmiTrDHI: Trickbox DHI Register [16] (+0x18)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
15:0	KmiTrDHI	R/W	Setup on KDATA from KCLK rising for Receive

KmiTrDSO: Trickbox DSO Register [16] (+0x1c)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
15:0	KmiTrDSO	R/W	Setup on KDATA to KCLK rising for Transmit

KmiTrDHO: Trickbox DHO Register [16] (+0x20)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
15:0	KmiTrDHO	R/W	Setup on KDATA from KCLK falling for Transmit

KmiTrREFCLK: Trickbox Reference Period Register [8] (+0x24)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:0	KmiTrREFCLK	R/W	Reference Clock Period

KmiTrRG: Trickbox RG Register [8] (+0x28)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:0	KmiTrRG	R/W	KCLK LOW from controller to KCLK LOW from Peripheral for request to send

KmiTrFFLAG: Trickbox SB Register [6] (+0x2c)

This register contains the fifo flag status.

Bits	Name	Read/Write	Function
5	KmiTrRXFF	R	Receive FIFO Full
4	KmiTrRXFE	R	Receive FIFO Empty
3	KmiTrRXFH	R	Receive FIFO Half Full
2	KmiTrTXFF	R	Transmit FIFO Full
1	KmiTrTXFE	R	Transmit FIFO Empty
0	KmiTrTXFH	R	Transmit FIFO Half Empty

KmiTrCHECKPIN: Trickbox Check Pin Register [5] (+0x30)

This register provides the ability to read certain output lines of the KMI (input lines of the Trickbox)

Bit	Name	Read / Write	Function
4	KmiINTR	R	State of KMIINTR pin of the KMI
3	KmiRXINTR	R	State of KMIRXINTR pin of the KMI
2	KmiTXINTR	R	State of KMITXINTR pin of the KMI
1	KDATAIN	R	State of KDATA pin of the KMI
0	KCLKIN	R	State of KCLK pin of the KMI

KmiTrTIMOUT: Trickbox Time Out Register [5] (+0x34)

This register provides the way to generate the Time Out at particular bit position.

Bit	Name	Read / Write	Function
4	TimeOutEn	R/W	Enables Time Out
3:0	TimeOutBit	R/W	Number of Bit where time out has to be done

KmiTrCLKDIV: Trickbox Reference Clock Divisor Register [8] (+0x38)

All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:0	KmiTrCLKDIV	R/W	Reference Clock Divisor

KmiTrMODEREG: Trickbox Clock Mode Register [4] (+0x3c)

This register provides the ability to control the KMI Reset and reference clock.

Bit	Name	Read / Write	Function
3	REFCLKEn	R/W	Internally generated clock is Enabled
2	PCLKEn	R/W	PCLK is being enabled with this bit for routing to the KMIREFCLK.
1	ClockStatus	R/W	If 1, The PCLK is being routed to REFCLK Output. In case of 0, internally generated clock is being passed.
0	Reset	R/W	Value programmed in this bit decides the level on the nKMIRST Line.

KmiTrTIMESTAT: Trickbox Timing Error Status Register [2] (+0x40)

Bit	Name	Read / Write	Function
1	KmiTrDSOErr	R/W	DSO Timing Error Bit. Cleared On Write to this register.
0	KmiTrDHOErr	R/W	DHO Timing Error Bit. Cleared On Write to this register.

6.3 Free-running Functional Verification Tests

The following sections describe all the free-running test cases that were run during the validation of the KMI. These tests use the KMI Trickbox .

6.3.1 Register Tests

Reset Value Tests

After a reset, each of the following registers are read to check if they contain the specified reset value.

Register Name	Reset Value
KMISTAT	0x43
KMICKDIV	0x00
KMICR	0x00

Read/Writeability of the writeable registers.

A data pattern is written to each of the writeable registers KMICR and KMICKDIV .

The register contents are read back to see if it contains the written value. If the values are the same then the register is declared to be writeable and readable and the test passes.

6.3.2 Transmission Test

In the transmit test the KMI and the Trickbox are enabled , a byte is written to the data register of the KMI and then allowed to run till the byte is completely transmitted. The data byte is then read from the Trickbox and checked for error free reception. The flags in the KMISTAT (flag register) are checked to see if they have the expected values before , during and after transmission of the byte. The interrupt lines are checked to see for expected values before and after transmission of the byte. This test is run for both PS2/AT and Legacy mode with the Trickbox sending the ACK pulse in case of PS2/AT mode.

6.3.3 Reception Tests

In the receive test, the KMI and Trickbox are enabled, a byte is written to the data register of the Trickbox and then allowed to run till the byte is received. The data byte is read from the KMIDR register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte.

6.3.4 Transmission Series Test

In each transmit test the KMI and the Trickbox are enabled, a byte is written to the data register of the KMI and then allowed to run till the byte is completely transmitted. The data byte is then read from the Trickbox and checked for error free reception. The flags in the KMISTAT (flag register) are checked to see if they have the expected values before, during and after transmission of the byte. The interrupt lines are checked to see for expected values before and after transmission of the byte. This test is repeated sixteen times with different data values. This test is run for both PS2/AT and Legacy mode with the Trickbox sending the ACK pulse in case of PS2/AT mode.

6.3.5 Reception Series Test

In the receive test, the KMI and Trickbox are enabled, a byte is written to the data register of the Trickbox and then allowed to run till the byte is received. The data byte is read from the KMIDR register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. This test is repeated sixteen times with different data values.

6.3.6 Transmission Reception Alternating Test

In this test the KMI and the Trickbox are enabled, a byte is written to the data register of the KMI and then allowed to run till the byte is completely transmitted. The data byte is then read from the Trickbox and checked for error free reception. The flags in the KMISTAT (flag register) are checked to see if they have the expected values before, during and after transmission of the byte. Next a byte is written to the data register of the Trickbox and then allowed to run till the byte is received. The data byte is read from the KMIDR register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. This test is repeated sixteen times with different data values with alternate transmission and reception by the KMI. This test is run for both PS2/AT and Legacy mode with the Trickbox sending the ACK pulse in case of PS2/AT mode.

6.3.7 Transmission Reception Random Test

In this test the KMI and the Trickbox are enabled, a byte is written to the data register of the KMI and then allowed to run till the byte is completely transmitted. The data byte is then read from the Trickbox and checked for error free reception. The flags in the KMISTAT (flag register) are checked to see if they have the expected values before, during and after transmission of the byte. This transmission process is repeated with different data value from one to four times. Next a byte is written to the data register of the Trickbox and then allowed to run till the byte is received. The data byte is read from the KMIDR register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. This receive process is repeated with different data value from five to two times. The whole test is repeated four times as follows:

Loop 1: transmit with different data once and receive with different data five times.

Loop 2: transmit with different data 2 times and receive with different data four times.

Loop 3: transmit with different data 3 times and receive with different data three times.

Loop 4: transmit with different data 4 times and receive with different data two times.

This test is run for both PS2/AT and Legacy mode with the Trickbox sending the ACK pulse in case of PS2/AT mode.

6.3.8 Receive State Test

The KMI and the TrickBox are both enabled. Write a data pattern to the TrickBox. Poll the KMI status register until the data reception has been detected. Wait in a loop without reading the data out of the Receive data register. Write one byte to the KMI. The KMI should leave the intermediate wait state and commence transmission of the written byte. Poll the KMI status register to ensure that it returns to the wait state after the transmission is over. Now, read the received data byte out of the receive register. The transmitted data is verified by reading it out of the TrickBox receive register.

6.3.9 Transmit State Test

In this test a data pattern (0x55) is written to the KMI. This is immediately followed by another write to the transmit register with another data pattern (0xAA). The Trickbox interrupt status bits are polled until they indicate that a byte has been received. It is verified that the 0x55 pattern is received by the Trickbox. A third data pattern 0xFF is written to the KMI and waited until received by the Trickbox. The data pattern received by the Trickbox is 0xFF and not 0xAA. This indicates that the KMI transmit register ignores any write to it once the register is full. This test is run for both PS2/AT and Legacy mode with the Trickbox sending the ACK pulse in case of PS2/AT mode.

6.3.10 Error Tests

These tests check the error handling functionality of the KMI. Different types of errors are intentionally introduced into the data stream of the Trickbox. The various error detection tests are described in the following sections.

Parity Error Check Tests

In this test, a data frame with a parity error is driven on the KMIDATA line by the Trickbox. The parity error can be introduced by setting the KMI_FORCED_PARITY_ERR bit in the KMICR register of the Trickbox. The data byte is received by the KMI and the status register is checked to see if the parity error bit is set or not. If the parity error bit is not set in the KMISTAT then the test fails.

Width Measurement Tests

In this test, different clock division values are written to the KMI and Trickbox. In the Trickbox, the KMI_DATA_WIDTH and KMI_KCLK_WIDTH bits are enabled. In each transmit test the KMI and the Trickbox are enabled, a byte is written to the data register of the KMI and then allowed to run till the byte is completely transmitted. During the same time KCLK and KDATA widths are measured and if they are not in the specified limits then corresponding errors flags are set in the Trickbox status register.

6.3.11 Timeout Tests During Reception

This test uses the test registers of the KMI to reduce the simulation run time. The KMI second stage register and the first stage register are bypassed. The KMI is enabled and the flags in the KMISTAT (flag register) are checked for expected value. Then the KmiTrTIMOUT register is being programmed with the Time Out Enable and Bit Number where Time Out has to be generated. The Trickbox holds its KCLK line at programmed bit value in the KmiTrTIMOUT register. The KMI and Trickbox are then programmed in normal mode after 256, 8-Mhz clock periods. The Trickbox is enabled and a byte is written to the data register of the Trickbox and then allowed to run till the byte is received. The data byte is read from the KMIDR register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. This test effectively checks correct recovery of the KMI after a timeout condition has occurred.

6.3.12 Timeout Tests During Transmit

This test uses the test registers of the KMI to reduce the simulation run time. The KMI second stage register and the first stage register are bypassed. The KMI is enabled and the flags in the KMISTAT (flag register) are checked for expected value. Then the KmiTrTIMOUT register is being programmed with the Time Out Enable and Bit Number where Time Out has to be generated. Now, a data byte is written to KMI data register for starting transmission from the KMI. The Trickbox holds its KCLK line at programmed bit value in the KmiTrTIMOUT register. The KMI and Trickbox are then programmed in normal mode after 256, 8-Mhz clock periods. The Trickbox and Kmi are enabled and then allowed to run till the byte is transmitted. The data byte is read from the Trickbox data register followed by the status from the KMIRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. This test effectively checks correct recovery of the KMI after a timeout condition has occurred.

6.3.13 Scan Mode Test

This test is for validating the KMI scan test hold input.

Two known data patterns are written to two writeable registers in PCLK Domain. A test reset is asserted by setting and clearing the TESTRST (test reset) bit in the KMITCR (test control register) of the KMI. After this test reset, the two writeable registers should be reset to their reset values (the registers chosen in this case, KMICR and KMIClkDiv, both have reset values of 0x00).

Now the two known data patterns are written to the two registers again. The SCANMODE pin is driven high through the Trickbox. The test reset cycle is again conducted (that is the TESTRST bit is asserted and deasserted through the KMITCR register). This time the two registers KMICR and KMIClkDiv should retain the two known data patterns.

For testing the register in the REFCLK domain an event has been created by writing the data to the data register. This will change the KMIIR Register. A test reset is asserted by setting and clearing the TESTRST (test reset) bit in the KMITCR (test control register) of the KMI. After this test reset, the KMIIR should be reset to their reset values.

Now the same procedure is being repeated for SCANMODE pin 'HIGH'. The test reset cycle is again conducted (that is the TESTRST bit is asserted and de-asserted through the KMITCR register). This time the KMIIR should retain the data pattern.

6.4 Clock-enabled Production Tests

The KMI HDL can be configured such that the internal synchronisers between the PCLK and the KMIREFCLK domains are bypassed under the control of the Bypass HDL generic / parameter. In order to test the KMI functionality in either of these modes, a similar variable has been introduced in the Kmi_prod.c production level test code. Its usage is as follows:

The KMI production test code can also be configured using three other parameters viz DivBypass, DIVVAL and TYPE. If the DivBypass parameter is set to 1, the internal Pulse8MHz signal is continuously held high. This makes the Pulse8MHz signal independent of the DIVVAL parameter. If the DivBypass is set to 0, the period of the Pulse8MHz signal is determined by the parameter DIVVAL. DIVVAL mirrors the internal clock-prescaler and can assume values from 0x0 to 0xF. The parameter TYPE decides the type (PS2/AT or Legacy) of Keyboard being tested. By default, the TYPE is set to PS2/AT type (for which TYPE = 0) interface.

6.4.1 Reset Value Tests

The KMICR, KMISTAT, KMIDATA, KMICKDIV, KMITCER, KMITCR, KMITMR, KMITISR, KMITOCR, KMISTATE, KMITIMERSTG1, KMITIMERSTG2 and KMITIMERSTG3 registers are read immediately after reset to verify that they initialise to the values mentioned in the specifications.

6.4.2 Pattern Read / Write Tests

The Read / Writeable registers KMICR, KMITCR, KMITMR and KMITISR are written with patterns of 0x55 and 0xAA. Data is read back and compared with the expected pattern.

6.4.3 Read-Only Register Tests

The KMISTAT, KMITOCR, KMISTATE, KMITIMERSTG1, KMITIMERSTG2 and KMITIMERSTG3 registers are written with a pattern that is different from the reset value. Data is read back from the same register. An error is flagged if the read data is different from the reset value.

6.4.4 Reset Test Using Test Reset

A test value is written to all writeable registers. A reset is caused using Test Reset. All test registers should return the Test value when read, but other registers should return the reset values mentioned in the specifications.

6.4.5 Timer- Counter Test

This test checks the timers / counters both in the normal mode and the nibble mode. The KMITIMERSTG2 counter is tested by bypassing the KMITIMERSTG1 counter and the KMITIMERSTG3 counter is tested by bypassing both the KMITIMERSTG1 and KMITIMERSTG2 counters. Prior to starting the test, all the counters are placed in nibble mode. The test is conducted in the REG CLK mode. The counters are initialised by test reset and incremented by accessing the KMITCER register. The value of the counter is read from KMITIMERSTG1, KMITIMERSTG2 and KMITIMERSTG3 registers. During the test, the 64 usec and 16 msec taps of the counter are checked from the KMITIMERSTG2 register.

Then, all the counters are checked in normal mode as well.

6.4.6 Transmit Test

This test checks transmit functionality of the Kmi. This test also checks the relevant flags during the course of transmission via the KMISTAT register.

In this test, the KMITIMERSTG2 counter is fed straight from the pre-scaler, bypassing the KMITIMERSTG1 counter.

This test starts by checking whether the nKMICKEN signal remains low for a duration of 64us via the KMITOCR register. Each bit transmitted is then checked via the nKMIDATAEN bit of the KMITOCR register after a valid KMICKIN is supplied via the KMITISR register. This test checks the transmission of the start bit, the 8-bit data, parity and the stop bit for four data patterns.

6.4.7 Receive Test

This test checks receive functionality of the Kmi. This test also checks the relevant flags during the course of reception via the KMISTAT register.

In this test, the KMITIMERSTG2 counter is fed straight from the pre-scaler, bypassing the KMITIMERSTG1 counter.

The receive data – start, 8-bit data, parity, stop bit - is simulated by toggling the KMIDATAIN bit of the KMITISR register. After reception, the data and its corresponding parity bit is read from the KMIDATA and the KMISTAT registers respectively. This test simulates reception of four data patterns.

6.4.8 Transmit and Receive Test

This test checks the operation of the transmitter and the receiver state machine.

The following checks have been done during the course of this test.

Simultaneous reception and transmission was simulated to verify that the transmission takes priority by reading the state of the state machine via the KMISTATE register.

Correct transmission is verified via the nKMIDATAEN bit of the KMITOCR register after a valid KMICKIN is supplied via the KMITISR register.

After the transmission, the reception of a byte from the Keyboard is simulated. It is verified that the state-machine remains in the LOCK state until the received data is read out of the KMIDATA register.

After this reception, transmit one more byte. At the end of the transmission, check that the state machine once again enters the LOCK state, since the previously received byte has not yet been read. Now, read the received byte to check whether the state machine goes to the RESET state. Then, the KMICKIN line is pulled high to bring the state machine from RESET to IDLE state.

In the IDLE state, simulate the condition of reception of a byte from the Keyboard with a wrong stop bit and wrong parity bit to check whether the byte is discarded and the wrong parity bit still stored in the KMISTAT register. At the end of the reception, the state machine should return to the RECOVER state (rather than the LOCK state) and then goes to IDLE state to start another transmission or reception after the KMICKIN line has been pulled high. Simulate the condition of reception of another byte from the Keyboard with a valid stop and parity bit. At the end of the reception, the data byte is not read immediately from the Keyboard.

Place the KMITIMERSTG3 counter in bypass mode prior to conducting the TX and RX time out checks. Write a byte into the KMIDATA register to start a transmission and during transmission, pull the KMICKIN line low and keep it low for a pre-determined time so that the timeout condition occurs at TXDONE state. The time out condition is verified by checking for a transition to the RECOVER state via the KMISTATE register. At RECOVER state, pull KMICKIN line high to bring the state machine to LOCK state as the previously received data byte has

not yet been read from KMIDATA register. Now, the previously received data byte is read from the KMIDATA register and the state machine goes to RESET state. This is verified by checking the state of the state machine through KMISTATE register. Then, the timeout test exercised in all other possible TX states.

The timeout test is conducted for the receive side as well.

The transmission and reception disable is conducted in all states of the states machine by disabling KmiEnable signal.

During both reception and transmission, the relevant flags are checked through the KMISTAT register and state of the state machine via the KMISTATE register. Invalid KMICLKIN pulses are generated to check whether they are getting ignored by the internal logic.